Benchmarking AI Models Efficiently

Siomara van Straalen*

14 September 2026; revised 10 November 2026

Abstract

Benchmarks that score several AI systems, from single models to multi-agent debate set-ups [Du et al., 2024, Liang et al., 2024], against one another with language-model judges, in the tradition of DRACO [Zhong et al., 2026] and judge-based evaluation [Zheng et al., 2023], are rigorous and expensive. In a faithful run, where each answer is judged separately, without batching, the number of judge scorings is task count times system count times judge count times repeated-scoring count times regeneration count times configuration-setting count. An interview study of nineteen practitioners describes evaluation practice "spanning informal 'vibe checks' to organizational meta-work" [van der Maden et al., 2026]; we take the cost of the rigorous end of that range to be a main reason such comparisons are run rarely or once, while the systems being compared keep changing. This paper writes the cost of a DRACO-style benchmark as an explicit function of its design parameters and derives six simplifications, each with a closed-form reduction factor and a statement of the guarantee it gives up (for batched judging, of what it adds). One result runs against intuition: judge count is the wrong place to economise, since fewer judges save little wherever generation dominates the budget and remove the evidence of whether the result depends on which judge was used. The combined reduction, a ratio of total costs, is a weighted harmonic mean of a generation-side and a judging-side factor; the naive product of per-component factors overstates the saving, by a factor of about six to thirty in the configurations evaluated here. In the worked instance, with the cheap setting kept, the combined reduction is roughly two orders of magnitude at that instance's setting-cost ratio and assumed task floor, a third of that at the floor its own power inputs give, and materially less at the ratios we measured. The appendices record the published runs and instantiate the algebra on current commercial terms and on measurements made for this paper. A closing protocol runs the full benchmark once and each simplification beside it, so that the fidelity cost of every simplification is measured against its predicted saving. The author is not affiliated with the authors of DRACO; this is independent work that builds on their published benchmark.

*Correspondence through the paper's repository, https://github.com/SiomaraDAILFT/AI-Benchmarking. Text of this paper: © 2026 Siomara van Straalen, licensed under CC BY-NC 4.0 (free to share and adapt with attribution for non-commercial use; commercial use requires a licence from the author). Accompanying code: PolyForm Noncommercial License 1.0.0.

Contents

1 Introduction **3**

2 DRACO and Related Multi-System Evaluation Approaches **4**
2.1 DRACO . . . 4
2.2 Related and competing multi-system approaches . . . 4

3 Merits and Costs of the DRACO Protocol **6**

4 A General Cost Model **7**

5 Six Cost-Reducing Simplifications **9**
5.1 S1: Reducing the configuration-setting sweep . . . 9
5.2 S2: Reducing answer regenerations . . . 10
5.3 S3: Reducing judge repeat scorings . . . 11
5.4 S4: Batching judging across systems . . . 11
5.5 S5: Judge count, and the case against reducing it . . . 13
5.6 S6: Reducing task count, bounded by statistical power . . . 14

6 Cumulative Effect **15**

7 Statistical Constraints on the Simplifications **16**
7.1 Test, aggregation, and multiple comparisons . . . 16
7.2 Power and the S6 floor . . . 17
7.3 Blind and calibrated scoring . . . 18
7.4 Data retention . . . 19

8 Empirical Validation of the Reduction Factors **20**

9 Limitations and Open Questions **22**

10 Conclusion **23**

A Published DRACO-Style Runs **24**
A.1 The benchmark . . . 24
A.2 Published scores . . . 24
A.3 Third-party trackers . . . 25

B A Worked Cost Instance at Current Commercial Terms **27**
B.1 Prompt-caching terms . . . 27
B.2 The setting-cost ratio α . . . 27
B.3 Call counts for a modest configuration . . . 27
B.4 The batching factor β . . . 28
B.5 The combined reduction . . . 28
B.6 The task floor n^* . . . 28

C Measurements Made for This Paper **29**
C.1 Judging-side premises (experiment E2) . . . 29
C.2 The setting-cost ratio α (experiment E1) . . . 30
C.3 Full method for experiment E1 . . . 31
C.4 Why the two measurements of α differ . . . 33
C.5 Competition problems as a task-family test (experiment E3) . . . 35

1 Introduction

A benchmark that compares several AI systems or configurations (individual models, ensembles, retrieval-augmented pipelines, multi-agent panels) against one another on a shared task set, scored by one or more language-model judges against a written rubric, is now a standard methodology. It sits beside broad multi-scenario evaluations scored mainly by automatic metrics, such as HELM [Liang et al., 2023], and crowdsourced pairwise-preference leaderboards scored by human votes [Chiang et al., 2024], and it underlies the automatic, judge-scored benchmarks built from those leaderboards or checked against them [Li et al., 2024b, Dubois et al., 2024]. DRACO [Zhong et al., 2026] is a recent, well-specified instance and the running example here. The author is not affiliated with DRACO's authors, and they have not reviewed this work. The literature on language models as judges, surveyed by Li et al. [2024a] and Li et al. [2025], supplies what the judging-side simplifications cost, and work on test-time compute, nondeterminism and benchmark compression supplies the rest (Section 5); the subject here is the cost of the benchmark as a whole. The methodology is powerful for the same reasons that it is expensive to run faithfully. Task sourcing from real usage, several judges, repeated scoring to separate judge noise from differences between systems, and enough tasks to reach statistical power each add expense and running time, and each of them matters to the result.

That expense is the obstacle addressed here. Van der Maden et al. [2026] found practitioners' methods ranging from informal "vibe checks" to organisational meta-work, with a gap between gathering evaluation data and acting on it; whether cost is what keeps teams from the rigorous end of that range is not something their study, or any other we know of, measures. We state it as a premise. Money is also only part of the expense. A rigorous multi-system evaluation takes time to set up: tasks and rubrics have to be written, several providers' interfaces connected, the judging blinded and randomised, every raw score retained and the statistics computed. The algebra below addresses the first cost; for the second, the tooling that runs the protocol of Sections 4–7 is published with this paper.[1] The set of systems worth comparing does not stay fixed. New models appear, existing systems are reconfigured, and a comparison that was current one quarter is stale the next. A methodology that can be afforded once a year cannot support decisions that are made continually. The question, then, is which parts of a rigorous protocol can be reduced, by how much, and at what cost to what the result can claim.

Section 2 describes DRACO and its neighbours, and Section 3 what DRACO does well and what it costs. Sections 4–6 give the cost model, the six simplifications and their combined effect; Section 7 the statistical constraints that bound them; Section 8 a protocol for checking predicted savings against measured fidelity loss; Section 9 the open questions. The appendices give the concrete figures: published DRACO-style runs (Appendix A), a worked instance at current commercial terms (Appendix B) and the measurements made for this paper (Appendix C).

[1] `https://github.com/SiomaraDAILFT/AI-Benchmarking`: the paper, a reference implementation of its equations with the tests that reproduce its worked figures, the protocol runner and the measurement scripts.

2 DRACO and Related Multi-System Evaluation Approaches

2.1 DRACO

DRACO (Deep Research Accuracy, Completeness, and Objectivity) is a benchmark for deep-research systems published in 2026 by the developer of one such commercial system [Zhong et al., 2026, Perplexity AI, 2026a]. Its tasks are drawn from anonymised real usage of that system: de-identified user requests, first pre-filtered to a high-difficulty stratum, then rewritten, augmented and filtered into a pool from which 100 items spanning 10 domains were sampled in proportion to that system's mix of domains and reviewed by its developer's domain experts; the authors describe the tasks as "anonymized, open-ended and complex, objectively evaluable, and representative". Each task carries an expert-written scoring rubric of several tens of criteria, graded on four dimensions (factual accuracy, breadth and depth of analysis, presentation quality, and citation quality) by a language-model judge, with the grading repeated and averaged. The benchmark paper gives no sample-size or statistical-power guidance and reports only the standard deviation of its scores across five grading runs; the only other public precision figure we could find is a standard error, which a third-party run of the benchmark reports on its headline difference [Anand, 2026]. Appendix A records the published figures.

Selecting tasks that are "objectively evaluable" is itself a form of selecting for gradeability, so the published task set is a curated sample from a filtered stratum rather than an unfiltered draw from real usage. The design still supports a weaker claim: each question began as a real user's request made for a real purpose, not as something a benchmark author wrote to be easy to grade, although a language model in the authors' pipeline rewrote it to remove personal details and reduce ambiguity, and added context and scope partly to keep the evaluation criteria consistent, before it entered the set.

2.2 Related and competing multi-system approaches

Leaving aside single models, which are also scored on DRACO, four structurally different kinds of system have reported DRACO scores or positioned themselves against it (Appendix A lists the published instances with their figures, judges and dates). First, *synthesis ensembles*: several models answer independently and a further model synthesises the answers; one API aggregator reports such ensembles, including a panel of inexpensive models, outscoring each constituent model alone [OpenRouter, 2026]. Second, the *benchmark author's own product*, whose scores should be read as vendor figures and not as independent confirmation [Perplexity AI, 2026b]. Third, a *multi-model research assistant* from a large software vendor, whose published run of the benchmark reports a statistically significant improvement over a single-model baseline under a different judge model from the aggregator's [Anand, 2026]. Fourth, *retrieval-centred* deep-research products that report the highest of the scores in Table 1, describe multi-step retrieval across specialised sources, and do not describe multi-model ensembling [Valyu, 2026].

Two further categories are sometimes conflated with these. A *router* [Not Diamond, 2024, Martian, 2023] selects a single best model per query; it does not combine multiple models' outputs and answers a different question from a multi-system panel. *Consumer parallel-chat products* [AllChat, 2026], which send one question to several assistants and return a consensus answer, are structurally closer to a panel but publish no benchmark score of their own: the one cited advertises 60% fewer factual errors and a depth score twice as high, without naming a baseline, and its benchmarks page states that it runs no internal benchmarks and aggregates other publishers' scores.

Because judge choice alone moves published DRACO scores by a wide margin (Appendix A), comparisons across those numbers should be read as directional at best. For that reason S5 (Section 5) argues against reducing the number of judges, and Section 9 explains that the interval of Section 7 is conditional on the judges used.

Combining several answers, from one model or from several, has its own evidence base: sampling several answers from one model and keeping the most consistent one [Wang et al., 2023b], ranking and fusing the candidates of several open models [Jiang et al., 2023], and layered mixture-of-agents designs, one of which, built only from open models, reached 65.1% on a judge-scored instruction-following benchmark against 57.5% for GPT-4 Omni [Wang et al., 2024]. That benchmark scores length-controlled win rates because its judge favours longer answers [Dubois et al., 2024], which is one reason Section 7 treats the judge as a source of error. Multi-agent debate over several rounds has been reported to improve factual accuracy and reasoning relative to a single instance [Du et al., 2024, Liang et al., 2024]. The same literature supplies a caution. Zhang et al. [2025] find that debate methods often fail to beat single-agent baselines such as chain-of-thought and self-consistency, even when they use more computation than the baselines, although debate improves when its agents are drawn from different models rather than all from one. Cemri et al. [2025], who built a failure taxonomy from 150 multi-agent traces read by human experts and had a language-model annotator apply it to more than 1,600, sort the failures into three groups: system design, misalignment between agents, and task verification; they also observe that the gains of multi-agent systems on popular benchmarks are often minimal. Gains this small are reasons to take the statistics of the scoring (Section 7) as seriously as the comparison itself.

3 Merits and Costs of the DRACO Protocol

Drawing tasks from real usage rather than composing them for the benchmark addresses a documented weakness of benchmarks built that way: their construct validity, the degree to which they measure what they are framed as measuring, is often unexamined [Raji et al., 2021], and dataset design and annotation reliability bound what a benchmark can show [Bowman and Dahl, 2021]; sourcing from real users is now an established design for that reason [Chiang et al., 2024, Lin et al., 2024]. A multi-dimension rubric graded per task is more informative than a pass/fail judgement or a generic quality prompt: Ye et al. [2024] show that "the fine-graininess of evaluation is crucial for attaining a holistic view of model performance and increasing the reliability of the evaluation", and Prometheus 2, a purpose-built evaluator model, judges against user-defined criteria, which its authors report existing open evaluators often could not do [Kim et al., 2024]. Repeated judge scoring separates judge noise from signal; S3 in Section 5 gives the evidence that the noise is real.

Against that, no sample-size or statistical-power methodology has been published for DRACO, as Section 2 noted. Judge choice moves absolute score levels: the benchmark's own authors report that one of their three judges "consistently assigned lower scores" [Zhong et al., 2026, Appendix A], and a study of 13 judge models on a short-answer knowledge benchmark finds scores that differ by judge while rankings mostly agree [Thakur et al., 2025]. Third-party trackers of DRACO results already disagree about which system leads and by how much (Appendix A); no figure should yet be taken from them.

The weakness we take to matter most for whether the methodology gets used at all is another: run rigorously, DRACO or any protocol with its structure is expensive and slow. The number of judge scorings in a single run, when each answer is judged separately, is the product of how many systems are compared, how many tasks, how many judges, how many times each judge re-scores an answer, how many times each answer is regenerated, and how many configuration variants (temperature, retrieval settings, reasoning-effort levels) are swept per system. Each factor is well motivated on its own, and Section 4 makes each explicit; together they make a faithful run expensive to repeat, since in such a run four of them (systems, tasks, regenerations and settings) multiply the whole cost and the other two multiply its judging term (Equations 1 and 2). Even with batched judging, three of them still multiply the whole cost (Equation 6).

4 A General Cost Model

We define the following parameters, the first six of them properties of the evaluation's design (the benchmark's, and for C the comparison's) and independent of any particular system under test, and the last two average costs per call:

- C: the number of systems or configurations under comparison.
- T: the number of tasks.
- E: the number of configuration-setting variants swept per system (for example, a minimum and a maximum reasoning-effort or compute-budget setting).
- R: the number of independent answer regenerations per (system, task, setting) triple, used to separate a system's true output distribution from a single unlucky or lucky draw.
- J: the number of judges (S5 explains why their errors cannot be assumed independent).
- K: the number of repeated scorings per (judge, answer) pair, used to separate judge test–retest noise from a real difference.
- g: the average cost of one generation call, that is, of obtaining one answer from one system, for one task, at one setting. It is treated as an observable, external quantity (a published API price, or a metered cost the system itself reports) and is not decomposed further; the framework is agnostic to how any system produces its answer.
- j: the average cost of one judge scoring one answer once, counted as one judging call, unbatched. DRACO scores each criterion independently [Zhong et al., 2026, Section 4.2], which can take one call per criterion; j then covers all the calls of one scoring, and the number of calls actually sent is each judging-call count below multiplied by the number of criteria.

Generation cost. Every system, at every setting, is queried R times per task:

$$G = C \cdot E \cdot R \cdot T \cdot g. \tag{1}$$

Judging cost, unbatched. Every one of the $C \cdot E \cdot R \cdot T$ generated answers is scored by every judge, K times each (a J with a subscript, here and below, is a judging cost; J alone is the number of judges):

$$J_u = C \cdot E \cdot R \cdot T \cdot J \cdot K \cdot j. \tag{2}$$

Judging cost, batched. If a judge is instead shown all C systems' answers to the same (task, setting, regeneration) triple in a single call and asked to score each, the C factor drops out of the call count, at the cost of a larger per-call price j':

$$J_b = E \cdot R \cdot T \cdot J \cdot K \cdot j'. \tag{3}$$

Decomposing j and j' into a fixed component (task description, rubric text, formatting instructions, paid once per call regardless of how many answers it covers) and a marginal component (the cost of reading and scoring one additional answer), $j = j_{\text{fixed}} + j_{\text{marginal}}$ and $j' \approx j_{\text{fixed}} + C \cdot j_{\text{marginal}}$, so batching saves

$$J_u - J_b = E \cdot R \cdot T \cdot J \cdot K \cdot (C - 1) \cdot j_{\text{fixed}}. \tag{4}$$

The saving is largest when the fixed overhead is large relative to the marginal cost of reading one more answer. That ratio can be checked empirically for a given rubric and task format; the systems compared affect it through the length of their answers, since a longer answer costs more to read.

Effect of prompt caching. Equation 4 assumes the fixed component is paid in full on every unbatched call. The largest commercial API providers now bill a repeated prompt *prefix* at a discounted cached-read rate, in some cases with a premium on the first, cache-writing call, a minimum

prefix length and a limited cache lifetime; output tokens are not cached [OpenAI, 2026a, Anthropic, 2026a, Google, 2026]. Appendix B records the terms in force when the providers' pages were read, on 11 September and on 26 and 27 October 2026. A task description together with a rubric of several tens of criteria can fall on either side of the published minimum prefix lengths: a 40-criterion rubric with its task and question came to about 1,005 tokens, just below one provider's 1,024-token minimum, and drew no cached tokens (Appendix C), so the prefix length should be checked.

The C calls in which one judge, in one repeat, scores the C systems' answers to one (task, setting, regeneration) triple naturally fall within a single cache lifetime, and the formula below charges a fresh cache write for each such group, that is, for every setting, regeneration and repeat. All $C \cdot E \cdot R \cdot K$ calls one judge makes on a task open with the same prefix, and repeated calls with an identical prompt were measured reading the cache (Appendix C). Where all the calls one judge makes on a task fall within one lifetime, the write is paid once per task and judge, and in the worked instance of Appendix B the cached judging-side factor F_J of Section 6 at $w = 1$ falls from 108 to about 69, and the cached reductions of Table 2 from 137–176× to about 101–161×. Writing c for the cached-read price as a fraction of the standard input price and w for the price of the first, cache-writing call as a multiple of the standard input price ($w = 1$ where no write premium is charged), the unbatched judging cost with caching is

$$J_u^{\text{cached}} = E \cdot R \cdot T \cdot J \cdot K \cdot \left[\left(w + (C-1)\, c\right) j_{\text{fixed}} + C \cdot j_{\text{marginal}} \right], \tag{5}$$

which reduces to Equation 2 at $c = w = 1$. We are not aware of a published treatment of prompt caching's effect on the cost of judged evaluation; batch prompting has been studied as a cost measure [Cheng et al., 2023] and batch-wise evaluation as a quality measure [Yuan et al., 2024], but caching enters only through provider pricing pages. Section 5 (S4) shows that caching weakens the case for batching considerably.

Total cost, using batched judging:

$$\text{Total} = G + J_b = E \cdot R \cdot T \cdot \left(C \cdot g + J \cdot K \cdot j'\right). \tag{6}$$

Equation 6 shows the problem: three of the six design choices (E, R and T) multiply the whole cost, and each of the other three multiplies one of its two terms. The number of generation calls is $C \cdot E \cdot R \cdot T$ and the number of unbatched judging calls is that quantity multiplied again by $J \cdot K$, so a configuration that looks modest on every individual dimension implies call counts in the thousands and tens of thousands respectively before any per-call price is attached to them (Appendix B evaluates one such configuration).

5 Six Cost-Reducing Simplifications

Each simplification below states its reduction factor in closed form and, separately, what it trades away. Nothing here recommends adopting all of them.

5.1 S1: Reducing the configuration-setting sweep

If E settings are swept per system and one setting's generation cost is α times a cheaper baseline setting's cost (for example, an extended-reasoning or higher-compute setting against a minimal one; we call α the setting-cost ratio), reducing to the single cheapest setting saves, with g_e the average cost of one generation call at setting e and $g_{\min}$ the lowest of them, a factor of

$$\left.\frac{\sum_{e=1}^{E} g_e}{g_{\min}}\right|_{E=2} = 1 + \alpha \tag{7}$$

on generation cost, and a separate factor of E on judging cost (fewer conditions to judge), if an answer costs the same to judge at every setting; where the expensive setting's answers are longer, that factor is above E when the cheap setting is kept and below it when the expensive one is. Equation 7 assumes the *cheap* setting is the one retained. If instead the expensive setting is retained, which for reasons given below is often the appropriate choice, the generation-side factor is $(1+\alpha)/\alpha$, which tends to 1 as α grows; S1 then saves almost nothing on generation once α is large, and only the factor E on judging, or less where the expensive setting's answers are longer. If the two settings cost the same ($\alpha = 1$), either choice saves the trivial factor of 2.

The generation-side case for S1 rests on α being large and on the cheap setting being the operating point of interest. Two pieces of evidence bear on α. The largest providers bill a reasoning model's internal "thinking" tokens as output tokens and expose the amount as a per-request setting, so a call at a high setting pays the same rate on whatever extra tokens it uses [OpenAI, 2026b, Anthropic, 2026d]. An independent measurement service that runs a fixed suite at each setting reports, for one frontier model, more than an order of magnitude more output tokens at the highest setting than at the lowest, with a substantial difference in measured quality [Artificial Analysis, 2025]; Appendix B gives the figures, and the same service publishes the token counts and cost of running its whole suite for the models and settings it evaluates [Artificial Analysis, 2026], so α on that suite can be looked up; on the benchmark's own tasks it has to be measured. On open-ended analytic prompts we found α between 1.75 and 3.18 by output tokens (1.74 to 3.14 by cost) for two frontier models, comparing the `high` setting, which is below `xhigh` and `max` for both models, with a lower one (Appendix C), an order of magnitude below that published figure; α differed between the two models and, for one of them, between these prompts and competition problems given a different instruction (Section C.5), and it may also depend on how each provider implements its effort setting (Section C.4).

What is lost: questions about the contrast between settings can no longer be answered, since only one setting is measured. A less obvious loss matters more. Measuring every system at its cheapest setting and reporting a ranking of the *systems* assumes that the order at the cheap setting predicts the order at the expensive one, and the evidence runs the other way. Guedes de Souza and Panisson [2026] vary the generation budget, the maximum number of output tokens, over seven levels from 64 to 4,096 for four models on three reasoning benchmarks and find that "model rankings reverse across budgets on all benchmarks" ($p < 0.01$), with 3–19% of items losing accuracy as the budget grows. The effect of extra compute is itself uneven. It can let a smaller model beat one 14× larger at equal compute, on problems the smaller model already solves some of the time [Snell et al., 2024], or one 5× larger [Wu et al., 2025], lift an open code model from 15.9% to 56% of tasks solved

with 250 samples [Brown et al., 2024], and at the same time lower accuracy on hard queries while raising it on easy ones [Chen et al., 2024]. In one automated-scoring task higher effort helped on average but its benefit "varied by model family", and the most accurate configuration ran at low effort [Frohn, 2026]. On closed-book knowledge questions, answered without search, extra compute "does not consistently improve accuracy and often leads to more hallucinations" across 14 reasoning models, although for GPT-5-mini with a search tool, closer to the conditions of DRACO's tasks, it raised accuracy from 41.2% to 53.0% and cut hallucinations from 16.6% to 8.0% [Zhao et al., 2026a]. Kim and Jung [2026] accordingly ask that medical studies of language models report the effort setting, its rationale and whether it was left at the vendor default, since differing settings can confound the gap between models when several are benchmarked.

The consequence for S1 is a scope condition. A comparison run at one setting answers the question "which system is best *at this operating point*", and its result should be reported with the setting named, as Guedes de Souza and Panisson [2026] recommend; it does not license a claim about the same systems at any other setting. Where the expensive setting is the one that would be used in practice, the appropriate reduced design measures at *that* setting alone and accepts the smaller generation-side saving of $(1+\alpha)/\alpha$. S1 removes the sweep; it does not change the operating point. Where both settings matter, the pilot of Section 8 should include a small two-setting subset and report the rank agreement between settings (Kendall's τ, or the sign of each pre-registered difference) before the sweep is dropped.

5.2 S2: Reducing answer regenerations

Reducing R regenerations to 1 saves a factor of R on both generation cost (Equation 1) and judging cost (Equations 2–5), since both scale linearly with R.

What is lost: any way of telling whether a single answer was a representative draw from the system's output distribution or an unusually good or bad one. That distribution is not degenerate even at temperature zero. An open-weight model (Qwen3-235B), sampled 1,000 times at temperature zero on the same prompt, returned 80 distinct completions, a consequence of server-side batching acting through floating-point non-associativity [He and Thinking Machines Lab, 2025], and in an open-weight reasoning model the same numerical cause gives greedy-decoding accuracy a standard deviation of up to 9 points across 12 combinations of batch size and the number and type of graphics processors, at 16-bit (bfloat16) precision on the 2024 American Invitational Mathematics Examination [Yuan et al., 2025]. At the level of evaluations, accuracy varied by up to 15 points across ten "deterministic" runs of five models on eight tasks [Atil et al., 2024]; between 48% and 76% of code-generation tasks had no test on which the programs from five repeated requests at the default temperature all gave the same output, with temperature zero reducing but not removing the effect [Ouyang et al., 2023]; the choice between greedy decoding and sampling shifts measured performance [Song et al., 2024]; and Madaan et al. [2024], measuring the variance across seeds and checkpoints, say their work "encourages practitioners to carefully factor in variance when comparing models". At $R = 1$ this within-system variance is not removed; it goes unmeasured and is absorbed into whatever difference the benchmark reports. Where S2 is applied, the decoding configuration should be fixed and stated, and the pilot of Section 8 should include a small multi-regeneration subset so that the size of the ignored variance is at least estimated.

5.3 S3: Reducing judge repeat scorings

Reducing K repeats to 1 saves a factor of K on judging cost only (Equations 2–5); generation cost is unaffected, since K only multiplies the scoring step.

What is lost: test–retest reliability, that is, whether a single judge's score on a given answer is stable when the judge is asked again with no memory of the prior scoring. The concern is documented for language-model judges. Stureborg et al. [2024] report low agreement between repeated samples of the same evaluator on the same item, alongside sensitivity to prompt differences that a human reader would consider immaterial; Schroeder and Wood-Doughty [2024] treat judge reliability in the classical psychometric sense, estimate it with McDonald's omega over repeated ratings, and conclude that "a single sample from the model's probability distribution can still be misleading"; and Thakur et al. [2025], across 13 judge models, find that even the best-aligned judges' scores "may still differ with up to 5 points from human-assigned scores", with a general tendency towards leniency. DRACO's own published protocol averages over five grading runs per answer and reports the resulting standard deviation [Zhong et al., 2026]; the third-party run also averages over five runs per question but reports a standard error of its headline difference instead [Anand, 2026]. A judge whose repeated scores on the same answer spread widely is indistinguishable, at $K = 1$, from a judge whose single score happened to be reliable; in a small measurement made for this paper, five scorings of one answer by one frontier judge spanned 17 points on a 100-point scale (Appendix C). Because judge repeats are inexpensive relative to generation wherever generation dominates the budget (S5 below gives the condition), and because the reliability they measure is mainly a property of the judge, although it can differ between the answers it scores, the economical course is to estimate it once, in the pilot, with $K > 1$ on a subset, and to set K for the simplified protocol from that estimate.

One further result bears directly on how many repeats are needed. Where the judging interface exposes the probabilities the judge assigns to each candidate score token, a single call yields the *expected* score over that distribution, and Wang et al. [2025] find that "taking the mean of the judgment distribution consistently outperforms taking the mode (i.e. greedy decoding) in all evaluation settings", that is, pointwise, pairwise and listwise. Repeated sampling at temperature one, from a judge that states its score without writing any reasoning first, is a Monte Carlo approximation of that same mean, so for such a judge, where score probabilities are available, one call replaces the mean of the K repeats for the purpose of averaging out sampling noise, though not their median, which Section 7.1 takes and which can differ from that mean where the judge's scores are skewed. A judge that writes reasoning or a justification before its score is different, and DRACO's judges write a short justification before each criterion's verdict [Zhong et al., 2026]. After such reasoning, Wang et al. [2025] find that the score distribution often sharpens, "making the mean judgment similar to the mode", as the model "commits to one instantiation of a reasoning trace"; one call's mean is then conditional on that one trace and does not average over the traces that repeated calls would sample. $K > 1$ therefore remains useful for position calibration (Section 7.3), for a judge that reasons before it scores, and where the interface offers no probabilities.

5.4 S4: Batching judging across systems

Showing a judge all C systems' answers to the same task in one call, instead of scoring each system's answer separately, reduces the judging term by the factor

$$\beta \equiv \frac{J_u^{\text{cached}}}{J_b} = \frac{w + (C-1)\,c + C\rho}{1 + C\rho}, \qquad \rho \equiv \frac{j_{\text{marginal}}}{j_{\text{fixed}}}, \tag{8}$$

with c and w the cached-read fraction and cache-write multiple of Equation 5. The batched call is priced here without a cache write; where caching is automatic, as at some providers, it pays the write too and the denominator becomes $w + C\rho$ (Appendix B). Without caching ($c = w = 1$) β is $C(1+\rho)/(1+C\rho)$: it approaches C as $\rho \to 0$ (the fixed overhead dominates the per-call cost, so batching amortises it across all C answers) and approaches 1 as $\rho \to \infty$ (the per-answer reading cost dominates, and there is nothing left to amortise). With caching and no write premium ($w = 1$), the corresponding limits are $1 + (C-1)\,c$ and 1: with one cache write per group of C unbatched calls (Equation 5) the batching factor cannot exceed $1 + (C-1)\,c$, which at the cached-read fractions currently published (Appendix B) stays well below C and grows by only c for each system added (1.4 at $C = 5$ and $c = 0.1$, against 5 without caching). The cost case for S4 is therefore weak wherever the judging prefix clears the provider's minimum and prompt caching is available, as it now is from the three largest providers (Section 4), from two of them by default on current models (Appendix B).

What is added rather than lost: a judge shown several systems' answers to the same task side by side has calibration context it would lack when scoring one answer in isolation. A method built on this, run iteratively in two stages over heterogeneous batches, has been measured to help: Yuan et al. [2024] find that scoring one sample at a time is "sensitive to prompt design" and shows "poor resistance to noise", and that their method, which presents a batch of samples in one prompt so that in-batch comparison serves as a second reference alongside the rubric, "outperforms state-of-the-art methods by 10.5% on Pearson correlations with only 64% API cost on average". The same direction appears in the comparative-assessment literature: pairwise comparison correlates better with human judgement than isolated absolute scoring in judging generated text, for moderate-sized open models in one study [Liusie et al., 2024] and, with a search over the pairwise comparisons, for GPT-3.5-turbo and GPT-4-turbo in another [Liu et al., 2024b]; and isolated scoring is the format in which G-Eval, a judge protocol its authors report as well ahead of earlier methods, reaches only a Spearman correlation of 0.514 with human raters on summarisation [Liu et al., 2023]. Batching is a candidate methodological improvement (Section 7.3) as well as a cost measure.

Batching has costs of its own. The judge may score *relative* to the batch instead of independently against the absolute rubric, even when instructed to do the latter. The distinction matters, because purely comparative judgements are more easily moved by features irrelevant to quality: Tripathi et al. [2025] find that pairwise preferences flip in about 35% of cases when a distractor feature is introduced, against 9% for absolute scores, and conclude that "absolute scoring is more robust to such manipulation". The protocol we recommend, an absolute rubric with the batch present only as reference, is the point between the two formats that the evidence favours. The judge can be held to it only by instruction, and instructions are imperfectly followed. A C-way listwise prompt is a format language models handle unreliably: Qin et al. [2024], ranking documents for search, argue that "off-the-shelf LLMs do not fully understand" pointwise and listwise ranking formulations, and Cheng et al. [2023] report that batched-prompt accuracy depends on "the number of samples in each batch and the complexity of tasks". And the longer multi-answer prompt exposes the judge to the documented degradation in how language models use information placed in the middle of a long context [Liu et al., 2024a], and to position bias, which Section 7.3 treats with its sources. Anonymised, order-randomised presentation with a small C reduces these costs without removing them; under caching the trade is made for calibration, since the cost saving is small.

5.5 S5: Judge count, and the case against reducing it

We recommend against this one. Fewer judges are cheaper, but not by much when generation dominates the budget. From Equation 3, the absolute saving from reducing J judges to 1 is

$$\Delta_J = E \cdot R \cdot T \cdot (J-1) \cdot K \cdot j', \tag{9}$$

which is a fraction Δ_J/Total of the whole budget (Equation 6) equal to

$$\frac{(J-1) \cdot K \cdot j'}{C \cdot g + J \cdot K \cdot j'}. \tag{10}$$

Whenever generation cost dominates judging cost, that is, whenever $C \cdot g \gg J \cdot K \cdot j'$, this fraction is small. A cheaper judge is not enough on its own for that: $J \cdot K$ is often larger than C (15 against 5 in the worked instance of Appendix B), so a generation call has to cost several times a batched judging call, which scores all C answers at once. Where generation dominates, reducing J saves little in absolute terms, and in any case it removes the one thing only judge count provides, evidence about whether the result depends on which judge was used. A single judge still yields a confidence interval across *tasks*; what it cannot yield is any estimate of judge-to-judge variation, so the resulting interval is conditional on that one judge and treats its idiosyncrasies as ground truth (Section 9 returns to this in the language of generalisability theory). Judge idiosyncrasy is well documented: the same answer receives different scores from different judge models, judges differ in leniency, and their absolute levels are not interchangeable [Thakur et al., 2025, Zheng et al., 2023]; DRACO's own authors found that one of their three judges "consistently assigned lower scores than the other two" [Zhong et al., 2026].

Judge count buys independent evidence only if the judges' errors are independent. Kohli [2026] shows that a panel of nine frontier models from seven families, tested on three natural-language-inference datasets, delivered only about two effective votes' worth of information, because roughly three quarters of the panel's nominal independence was lost to shared errors on the same items; the panel's accuracy fell 8–22 points short of what independent voting would have achieved, and the best single model matched or exceeded the whole panel. Nor is diversity of provider a sufficient remedy. Across more than 350 models, Kim et al. [2025] find that "larger and more accurate models have highly correlated errors, even with distinct architectures and providers"; on one leaderboard dataset "models agree 60% of the time when both models err"; and they trace the consequence directly into evaluation with language-model judges. Judge independence has to be measured: inter-judge agreement, or an effective-sample-size estimate of the kind Kohli [2026] uses, should be reported so that a reader can see how much independent evidence the panel contains, a reporting standard Yang et al. [2026] also call for after showing that "an LLM-as-judge score can move even when the candidate responses stay fixed, simply because the evaluator has changed". On the aggregation side, methods that model the judges' shared confounder explicitly recover part of what correlation costs [Zhao et al., 2026b]; they presuppose that the per-judge scores were retained, which Section 7 requires in any case.

More judges need not cost more, either. Verga et al. [2024] find that a panel of several smaller models from different families correlates better with human judgement than a single large judge, exhibits less intra-model bias, and costs over seven times less. Under Equation 3 the relevant quantity is the product $J \cdot j'$: a panel of three inexpensive judges can cost less than one expensive judge while providing the judge-generalisability evidence a single judge cannot.

Before any dimension is reduced, find the term of Equation 6 it controls and reduce the one that is expensive.

5.6 S6: Reducing task count, bounded by statistical power

Reducing T saves linearly on both generation and judging cost, and it has an explicit consequence: below a computable floor n^* (Section 7.2), the benchmark no longer has the stated power to detect, at the stated significance level, the difference in mean score it was designed to detect. Fewer regenerations (S2) and fewer repeat scorings (S3) also cost power, since they raise σ, the standard deviation of the task-level paired differences, and with it n^* (Equation 14); S6 is the dimension that floor bounds directly, and the floor can be computed once a pilot has produced a variance estimate.

What is lost: statistical power alone, provided the reduced task set is representative of the full one. That proviso is where the evidence from the benchmark-compression literature is relevant. Maia Polo et al. [2024] show that "to accurately estimate the performance of an LLM on MMLU ... consisting of 14K examples, it is sufficient to evaluate this LLM on 100 curated examples"; Perlitz et al. [2024a] report that evaluation compute can often be reduced "by x100 or more" on HELM with "minimal loss of benchmark reliability", while warning that "a benchmark leader may change by merely removing a low-ranked model from the benchmark"; and Vivek et al. [2024] find that "1–30 anchor points outperforms uniform sampling and other baselines at accurately ranking models". Those figures do not transfer directly. The compressed benchmarks in that literature start from hundreds to tens of thousands of items, mostly multiple-choice (the first of those works also compresses AlpacaEval 2.0, 805 open-ended prompts scored by a judge), with a large pool of previously evaluated models from which item informativeness can be estimated; a DRACO-style benchmark starts from about a hundred open-ended tasks (DRACO has 100) and no such pool, so the curated-subset methods are available only once several rounds of retained raw data (Section 7) exist. And any informed selection of tasks must itself be fixed before the systems are run, or task selection becomes one more researcher degree of freedom [Simmons et al., 2011]. Until such data exist, S6 reduces T by pre-registered stratified sampling from the task domains, in proportion to each domain's share of the tasks and with at least one task from each (a sample smaller than the number of domains takes one task from each of that many domains, chosen with a fixed seed), and Equation 13 sets the floor. The endpoint towards which S6 develops is already described in that literature: adaptive item selection driven by an item response model, which Hofmann et al. [2025] show attains higher validity and lower variance on MMLU "with fifty times fewer items", and subset evaluation with confidence intervals of valid frequentist coverage, for which Wu et al. [2026] report effective-sample-size gains of up to five times over uniform sampling.

6 Cumulative Effect

The simplifications do not act on the same quantity. S1's factor $(1+\alpha)$, the $E = 2$ case of $\sum_e g_e/g_{\min}$ (Equation 7), acts on generation cost; S1's factor E, S3's factor K and S4's factor β act on judging cost only; S2's factor R and S6's factor T_{full}/n^*, the full protocol's task count over the floor, act on both. A single product of all of them applied to the total would credit the generation share with E, K and β, which only the judging share can deliver, and the judging share with $(1+\alpha)$, which only the generation share can deliver. The exact combined reduction is a ratio of totals. Take the full protocol to judge each answer separately, without batching, as DRACO does, so that its judging cost is $J_{\text{full}} = J_u^{\text{cached}}$ (Equation 5; J_u where caching is unavailable), let its generation cost be $G_{\text{full}} = G$ (Equation 1), and let s be the share of the full protocol's cost that is judging, $s = J_{\text{full}}/(G_{\text{full}} + J_{\text{full}})$. Define the two component factors

$$F_G = (1+\alpha) \cdot R \cdot \frac{T_{\text{full}}}{n^*}, \qquad F_J = E \cdot R \cdot K \cdot \beta \cdot \frac{T_{\text{full}}}{n^*}, \tag{11}$$

with β from Equation 8 ($\beta = 1$ if S4 is not applied), and with the first factor of F_G replaced by $(1+\alpha)/\alpha$ if S1 retains the expensive setting instead of the cheap one. The reduced total is $G_{\text{full}}/F_G + J_{\text{full}}/F_J$, so

$$\text{Reduction} = \frac{G_{\text{full}} + J_{\text{full}}}{G_{\text{full}}/F_G + J_{\text{full}}/F_J} = \left(\frac{1-s}{F_G} + \frac{s}{F_J}\right)^{-1}, \tag{12}$$

a weighted harmonic mean of F_G and F_J, which always lies between them. As $s \to 0$ the reduction tends to F_G. Generation dominates the full protocol's budget when a generation call costs much more than the $J \cdot K$ unbatched judging calls (15 in the worked instance) made on its answer, and in that case the judging-side simplifications S3 and S4 contribute little to the combined figure, provided s is also small against $E/(1+\alpha)$, the ratio of F_J without them to F_G when S1 keeps the cheap setting: in the worked instance ($\alpha = 20$, so that ratio is about 0.1) they still roughly double the combined reduction at $s = 0.1$, from about 97 to 175–195×.

The naive product $F_G \cdot E \cdot K \cdot \beta$ (equivalently, all factors applied to the whole budget) exceeds the exact reduction by the factor $(1-s)\,E \cdot K \cdot \beta + s\,(1+\alpha)$ when S1 keeps the cheap setting, which grows with $E \cdot K \cdot \beta$ and with α; when S1 keeps the expensive setting the second term is $s\,(1+\alpha)/\alpha$, which falls as α grows. In the worked instance this overstatement is 13 to 29 with the cheap setting kept and 6 to 27 with the expensive one (Appendix B gives the figures). At the setting-cost ratios we measured between the `high` setting and a lower one (1.74 to 3.14, or 1.75 to 3.18 by output tokens, Appendix C) it is smaller: 7 to 11 under cached-read pricing with no write premium ($w = 1$), falling below 10 once s passes about 0.22 to 0.26; 8 to 12 at a write multiple of $w = 1.25$ where the batched call pays no write, falling below 10 once s passes about 0.31 to 0.36; and 7 to 11 where caching is automatic and the batched call pays the write too, falling below 10 once s passes about 0.20 to 0.23.

And because F_G and F_J share the factors R and T_{full}/n^*, S2 and S6 are the only simplifications whose benefit is independent of s; they are also two of the three whose cost to validity the power calculation expresses, S2 through the regeneration term of Equation 14, S3 through the within-task variances in that term, and S6 through the floor n^* (Sections 5 and 7.2).

Appendix B evaluates Equation 12 for an illustrative configuration: roughly two orders of magnitude, with and without prompt caching, across plausible values of s, at that instance's setting-cost ratio α and its assumed floor $n^* = 60$, when S1 keeps the cheap setting (10–18× when it keeps the

expensive one, since F_G is then $1.05 \cdot 3 \cdot 3 = 9.45$); with the floor of 184 that its own power inputs give, S6 saves nothing and every figure falls to a third (45–74× with the cheap setting kept). At the values of α we measured on analytic prompts (Appendix C) the same configuration yields well under a hundredfold, because α enters F_G directly.

7 Statistical Constraints on the Simplifications

7.1 Test, aggregation, and multiple comparisons

The quantity of interest for each pre-registered comparison between two systems is the task-level paired difference in score, where a system's score on a task at a given setting is the median of each judge's K repeat scorings, then the median over the J judges, then the mean over the R regenerations, as the published runner computes it. Each comparison is made at one setting of each of its two systems: the setting named for that system or, where none is named, its first configured setting. On a rubric with criterion weights, such as DRACO's, the runner computes each scoring from the judge's verdicts: the sum of the weights of the criteria marked met, in which a criterion describing an error carries a negative weight, over the sum of the positive weights, as a percentage clamped to 0 to 100 and rounded to one decimal. On an unweighted rubric it takes the score line the judge writes or, where the judge writes none but gives a verdict on every criterion, the share of criteria marked met; a score line need not equal that share (Appendix C).

A paired design is used throughout because every system answers the identical task set, and task difficulty is shared across systems: as Miller [2024] put it, "because eval question scores are likely to be positively correlated, even across unrelated models, paired differences represent a 'free' reduction in estimator variance when comparing two models".

Where a judging call fails or gives no score, the published runner leaves that scoring out of the median; on a weighted rubric a judge's answer that leaves any criterion without a verdict the runner can read, such as a refusal or an answer cut off partway, gives no score, whatever score line it writes. Where a generation call fails, or every scoring of a regeneration is missing, the runner leaves that regeneration out of the mean; a task on which either system then has no score is left out of that comparison, and a comparison left with fewer than 3 paired tasks is not tested but still counts among the comparisons the Holm correction below is applied across.

The test is *one-sided* when the research question is directional ("does system A outperform system B"), and *pre-specified before any data are collected*: a paired permutation (sign-flip) test of the mean task-level difference, which is exact and distribution-free under the null hypothesis that the task-level differences are independent of one another and each symmetric about zero, as they are when the two systems' labels are exchangeable within each task, independently across tasks [Ernst, 2004]; read as a test of the mean, it also needs the differences to be symmetric about their mean, since skewed differences with a mean of zero lie outside that null and can be rejected more often than the stated level allows. The test does not rely on the central-limit approximation that Bowyer et al. [2025] show to understate uncertainty for evaluations with "fewer than a few hundred datapoints", which is the regime every task count in this paper falls into. At those counts the 2^n sign patterns of n tasks are too many to enumerate, so the p-value is estimated from a random sample of them (the published runner draws 100,000), and the test stays exact only if the observed pattern is counted with the sample, adding one to both the number of patterns at least as extreme and the number drawn. The published runner counts the observed pattern in this way, so its p-value is never zero. We recommend *against* testing the differences for normality and then choosing between a t-test

and a non-parametric alternative. The simulation evidence on whether that two-stage procedure inflates the Type I error rate is mixed [Schucany and Ng, 2006, Rasch et al., 2011, Rochon et al., 2012], and the recommendation does not depend on it: a branch chosen after seeing the data is a researcher degree of freedom of the kind that "dramatically increases actual false-positive rates" [Simmons et al., 2011], and a power calculation derived for one test (Section 7.2) is not valid for whichever test the branch selects.

A Holm correction [Holm, 1979], which controls the family-wise error rate at a chosen level a and is uniformly more powerful than Bonferroni, is applied across the specific, named comparisons of interest and not across every pairing the full $C \times C$ comparison matrix could generate, since that would over-correct for hypotheses that were never posed.

Cohen's d_z, the paired-samples effect size standardised by the standard deviation of the differences themselves and equal to $t/\sqrt{n}$ for the paired t-statistic over n tasks, is reported alongside every p-value; it is distinct from d_{av}, which ignores the correlation between the paired measurements [Lakens, 2013]. A statistically significant but small effect is a different finding from a significant, large one.

The confidence interval for the mean paired difference is obtained by inverting the same test: it holds every value that, subtracted from each task's difference, leaves the two-sided sign-flip test unable to reject at the chosen level, so it rests on the same independence assumption, on each difference being symmetric about the mean difference rather than about zero, and not on the central-limit approximation. The published runner computes this interval at the level $1-a$ for each pre-registered comparison and reports it with the mean difference, d_z and the p-value; with 5 tasks or fewer at $a = 0.05$ no value can be rejected and the interval is unbounded.

7.2 Power and the S6 floor

The task-count floor referenced in Section 5 is computed from the standard sample-size formula for paired samples,

$$n^* = \frac{(z_{a'} + z_\pi)^2 \sigma^2}{\delta^2}, \tag{13}$$

one-sided and rounded up to a whole number of tasks, with a' the Holm-corrected per-comparison significance level (the smallest threshold the correction procedure can demand across the pre-registered comparisons; using the uncorrected family-wise a here systematically understates the requirement), $z_{a'}$ and z_π the standard normal quantiles at $1-a'$ and at the intended power π, σ the observed standard deviation of task-level paired differences from a pilot run, and δ the minimum difference in mean score worth detecting, in score points (so δ/σ is the corresponding d_z), a decision made before the pilot runs. Statistics texts usually write the significance level as α and the power as $1-\beta$; we use a and π because α is already the setting-cost ratio of S1 and β the batching factor of S4. Equation 13 is the one-sided form of the standard normal-approximation sample-size formula; Miller [2024] derives the two-sided version for language-model evaluations and decomposes the variance, a result this paper relies on.

Writing ω^2 for the variance of the *true* task-level paired difference across tasks and σ_A^2, σ_B^2 for the within-task variance of each system's score in a single regeneration, the variance of an observed paired difference averaged over R regenerations, if the two systems' scores vary independently within a task (S4, which scores both answers in one call, does not ensure this), is

$$\sigma^2 = \omega^2 + \frac{\sigma_A^2 + \sigma_B^2}{R}, \tag{14}$$

which makes explicit an interaction the simplifications of Section 5 would otherwise hide: S2 (fewer regenerations) raises the second term and therefore raises n^*, and S3 (fewer repeat scorings) raises σ_A^2 and σ_B^2 themselves, since each regeneration's score then rests on fewer scorings, so that some of what S2 and S3 save is returned as additional tasks under S6. Miller [2024] also gives the stopping rule for regenerations, stated there for one model's score and carried over here to the paired difference: once $(\sigma_A^2 + \sigma_B^2)/R \ll \omega^2$, further regenerations "will have little effect on the standard error", and the pilot's estimates of ω^2, σ_A^2 and σ_B^2 are what determine where that point lies. Equation 13 is a starting point and no more: the test actually used (Section 7.1) is a permutation test on a finite sample, σ itself is estimated, and Bowyer et al. [2025] show that central-limit-based intervals are unreliable below a few hundred tasks. A simulation-based re-check of n^* against the permutation test, using pilot data, is therefore part of the protocol.

7.3 Blind and calibrated scoring

S4 (Section 5) bears on two properties of the judging protocol: blinding, which it leaves intact, and calibration context, which it trades against scoring each answer independently. **Blind scoring**, in which the judge is never told which system, setting or configuration produced the answer it is scoring, guards against a judge favouring a system because it has been told which system wrote an answer, or that one system is the "interesting" one, though not against a judge that recognises a system from the answer itself. The biases cited here mostly act through the answer itself, which blinding does not hide: Zheng et al. [2023] report verbosity bias and describe self-enhancement bias, a judge's preference for its own answers, though they say their data cannot determine whether the judges they tested have it; Philipp et al. [2026] find evaluators that "preferentially scored architectural siblings"; and Stureborg et al. [2024] report a preference for lower-perplexity, more familiar text. Blinding is compatible with either isolated or batched presentation. **Calibration context**, showing a judge several systems' answers to the same task together as in S4, addresses a different problem: a judge scoring one answer in isolation has no reference for the level of quality attainable on that specific task, which risks scale drift, where the same numerical score from two judges denotes different levels of quality because each is implicitly calibrated to a different internal reference point; Stureborg et al. [2024] document the related anchoring effect in multi-attribute judgements. Batched presentation brings its own, well-measured hazard: position bias. Wang et al. [2023a] show that the ranking of candidate answers "can be easily hacked by simply altering their order of appearance", to the point that a weaker open-weight model was preferred over a stronger proprietary assistant on 66 of 80 queries under one ordering, and Zheng et al. [2023] report the same effect. In a small measurement made for this paper, three answers graded together under an absolute rubric kept their ranking in two of three orderings, with the top two swapping places in the third, and an individual score moved by up to 17 points, but five repeated scorings of one answer under a fixed prompt also spanned 17 points, so with one call per ordering that measurement cannot separate position from test–retest noise (Appendix C). One of the three remedies Wang et al. [2023a] propose, balanced position calibration (scoring under several orderings and aggregating), multiplies the number of judging calls and so consumes part of what S4 saved. Batched, anonymised, order-randomised presentation under explicit absolute-scoring instructions, with the order varied across the K repeats of S3 so that the repeats double as position calibration, is our recommended way to obtain calibration context without fully sacrificing independence; the S4 discussion in Section 5 names the residual risk this does not eliminate. The published runner draws a random order for each batch from a fixed seed and rotates it one place per repeat, drawing a new order after as many repeats as the batch has answers (C when every system's answer is in it), so that the order is random even at $K = 1$ and

each such block of repeats shows every answer once in each position; the order shown is kept with each scoring.

7.4 Data retention

Every individual measurement (each judge's verdict on each rubric criterion, for each repeat scoring, each regeneration and each setting) should be retained at its finest grain and never averaged down before storage. Every reported summary statistic is then a view computed from that raw table, and the raw table remains the record. The published runner retains them so: for each judging call and answer it stores the verdict on each criterion, the score, the numbers of criteria met and unmet, the order in which the answers were shown and, for weighted rubrics, the per-dimension scores. Retaining the measurements, and not only their per-system summaries, is what makes it possible to later compute a confidence interval or a p-value for any pair of systems not among the originally pre-registered comparisons, without re-running a single generation or judging call, provided that any such post-hoc comparison is reported as *exploratory* and is not included in the same family-wise error control as the comparisons that were pre-registered before data collection. The distinction is the one Nosek et al. [2018] draw between prediction and postdiction: pre-registration "distinguishes analyses and outcomes that result from predictions from those that result from postdictions", and a retained raw table is what allows both to be reported, with the distinction intact, from the same run.

8 Empirical Validation of the Reduction Factors

Every reduction factor in Section 5 is a prediction about cost, not about the result. A simplification could save the predicted cost and change which system appears to win, or save the same amount and change nothing. Only running the full and the simplified protocol on the same systems and tasks tells the two cases apart, and we recommend doing so before a simplified protocol is adopted for routine use.

Phase 0: calibration. Validate the judging protocol itself against a small set of known-good and known-poor answers, and, where the domain is high-stakes, against a small human-rated gold subset, before it is relied upon for comparison data. Agreement statistics alone do not establish this. Philipp et al. [2026] find that the best of several language-model evaluators of medical answers reached inter-rater agreement with physicians consistent with the physicians' own ceiling (Cohen's $\kappa = 0.694$ against 0.709, with confidence intervals the authors call wide), yet "physicians scaled abstention with item difficulty, while frontier models assigned definitive scores in every case"; they conclude that "statistical alignment does not ensure clinical caution, and that evaluator independence requires explicit verification". The gold subset should be used to check not only whether the judge's scores agree with human raters but whether its confidence tracks item difficulty as theirs does, and the inter-judge agreement of S5 should be reported alongside.

Phase 1: full reference run. Run the complete, unsimplified protocol (E settings swept, R regenerations, J judges, K repeats, T_{full} tasks) on the systems under comparison. This run is performed once and serves as the reference against which every simplified variant is compared. It is also the pilot that the other sections refer to: it contains the two-setting, multi-regeneration and repeat-scoring subsets that S1, S2 and S3 ask for, and its data give the variance estimates from which n^* is set (Section 7.2) and the setting-cost ratio α on the benchmark's own tasks, rather than on the analytic prompts measured in Appendix C.

Phase 2: each simplification alone, and all of them combined, on the same systems, and on the same tasks or a sample of them. Run S1 alone, S2 alone, and so on, and the fully combined simplification from Section 6, each on the task set used in Phase 1, or, for S6 and the combined variant, on a pre-registered stratified sample of n^* of those tasks, drawn as S6 prescribes, with n^* computed at the variant's own R and K, since fewer repeat scorings also raise the within-task variances of Equation 14. Where that n^* is above T_{full}, as for the combined variant on the inputs of Appendix B (184 against 180 from its $R = 1$ alone, and more once its $K = 1$ is counted), the variant runs on all T_{full} tasks and is reported as below the stated power; so is any other variant whose n^* exceeds the number of tasks it runs on, such as S2 alone, at 184 on the same inputs. For each variant, record: whether the same system(s) rank highest; how far the point estimate and confidence interval shift relative to the Phase 1 reference; and whether the corrected p-value for the pre-registered comparisons crosses the significance threshold in the reference run but not in the simplified one, or vice versa. Such a crossing can happen by chance alone, more often the lower a variant's power, so one crossing does not by itself show that the simplification changed the result.

Rank agreement between a reduced and a full protocol is one criterion the benchmark-compression literature uses to certify a subset: Saranathan et al. [2025] select subsets of 1–20% of a benchmark that preserve full-benchmark rankings, reporting evaluation-cost reductions of 80–99%, and Perlitz et al. [2024a] score each design choice by its effect on ranking reliability. Kendall's τ is the statistic Perlitz et al. [2024a] use, and Saranathan et al. [2025] use Spearman's rank correlation. Phase 2 should report rank agreement as Kendall's τ over the systems under comparison, alongside the per-comparison outcomes. Comparing two protocols on the same systems is formally a benchmark-

agreement test, for which Perlitz et al. [2024b] show that "overlooked methodological choices can significantly influence" the result, in particular which systems are included and which protocol is treated as the reference, and propose standard practices. The full Phase 1 run is the reference here by construction, and the systems compared should be the full set under study and not a convenient subset.

Phase 3: fidelity cost per simplification, reported alongside cost saved. The useful output of this protocol is a table of cost saved against fidelity lost, per simplification, from which a future user of this framework can choose which simplifications to accept, and that choice may reasonably differ from the combination of Section 6; which trade-offs are acceptable is a decision the paper's formulas inform without making. Phase 2 measures each simplification alone and all of them together, not the combinations in between, so a different combination should itself be run against the Phase 1 reference before it is adopted.

9 Limitations and Open Questions

The generation-cost parameter g is treated as a single external number: in Equations 1 and 6, one average over every system and setting; in practice it may itself have meaningful variance (a system's answer length, and therefore its cost, can differ substantially task to task), which the framework absorbs by taking g as an average over tasks (Section 4) instead of modelling it separately. Because g enters the generation cost and the total only as a multiplier (Equations 1 and 6), both are right on average, but the framework does not say how far the actual spend can differ from them; a more complete treatment would model the distribution of g.

The batching saving in Equation 8 assumes a judge can meaningfully score C answers in one call without a quality cost from context length or attention dilution as C and the length of the answers grow; this should be checked empirically at the C and answer lengths the benchmark uses. The concern has independent support: Hsieh et al. [2024] find that "almost all models exhibit large performance drops as the context length increases", with only half of the models they test maintaining satisfactory performance at 32K tokens, and Liu et al. [2024a] document the position-dependent degradation to which a multi-answer prompt is exposed.

The nested measurement structure (R regenerations $\times$ J judges $\times$ K repeats, aggregated to one score per system per task) shrinks the regeneration and repeat variance by averaging, and leaves out the variance due to the choice of judges, at the aggregation step, before σ^2 is estimated for Equation 13. In the language of generalisability theory [Brennan, 2001], tasks, judges, judge repeats and regenerations are *facets* of the measurement, each contributing a variance component that a generalisability study estimates from a pilot in which every facet varies; the paired-difference interval of Section 7 generalises over tasks and over the regenerations and repeat scorings nested within each task, whose variation enters σ^2 through the within-task term of Equation 14. Over tasks it generalises only as far as they are a random sample of the tasks a conclusion is meant to cover, and DRACO's are a curated sample from a filtered stratum (Section 2). The interval is conditional on the particular judges, the setting at which each system is compared (S1), the decoding configuration and any unrandomised presentation orders used. That is the sense of the argument in S5: at $J = 1$ the judge facet's variance component is not zero but unidentifiable, and the same holds for the regeneration facet at $R = 1$ and the repeat facet at $K = 1$, which is why Section 5 argues for keeping several judges (S5) and asks for each of the other two to be estimated on a pilot subset before it is dropped. A mixed-effects model with crossed random effects for task and judge [Baayen et al., 2008], with regenerations nested within task and system but crossed with judge, and repeats nested within judge and answer, would propagate all components into a single interval, at the price of simplicity; without it, the interval of Section 7 stays conditional on the judge panel and aggregation rule. The raw table retained under Section 7 is what makes fitting such a model later possible without re-running anything.

The minimum difference δ worth detecting for a given comparison remains a domain-specific judgement, which the formulas in Section 7.2 take as an input. The full reference run of Section 8 is made once, on one set of systems, so reusing its σ, its α and its measured fidelity losses after that set changes assumes they hold for systems it did not include; α already differs between the two models of Appendix C.

10 Conclusion

Rigour in a DRACO-style benchmark is expensive by construction, because four of its six design choices multiply the whole cost when each answer is judged separately, as DRACO does, and three still do when judging is batched (Section 3). Most of that cost can be given back. The sweep over settings and the regenerations carry most of the saving, the sweep only when the cheap setting is the one kept and is the operating point of interest (S1), and the regenerations at some cost in power, since fewer of them raise n^* (S2, S6); the task count adds to it only as far as the power floor of Section 7.2 allows. Batched judging carries far less than its uncached arithmetic suggests once prompt caching is priced in (S4), and less still where reading and scoring an answer costs more than the prompt the answers share, as it did on the calls we measured (Appendix C). The judge count, the dimension intuition reaches for first, is the one to keep.

The combined reduction, a weighted harmonic mean of a generation-side and a judging-side factor, is roughly two orders of magnitude in the worked instance of Appendix B with the cheap setting kept and at its assumed task floor, a third of that at the floor its own power inputs give, and less at the setting-cost ratios we measured. What that buys is a benchmark that can be re-run for a fraction of the full protocol's cost whenever the set of systems changes, provided the reduced protocol is checked against the full one at least once (Section 8) and what that check measured still holds for the new systems (Section 9).

A Published DRACO-Style Runs

This appendix records the named, dated figures behind Sections 2 and 3. Each was checked against the primary page cited; where a primary page no longer resolves, the entry records this. All access dates are September 2026, except that the Microsoft post was re-checked at its new address on 25 October 2026, and the Valyu post was re-checked and the Claude Opus 5 system card [Anthropic, 2026c] and Perplexity's blog post on DRACO [Perplexity AI, 2026b] read on 26 October 2026.

A.1 The benchmark

DRACO was published by Zhong et al. [2026] (Perplexity AI; arXiv, 12 February 2026), with the task set released under an MIT licence [Perplexity AI, 2026a]. The task pipeline began from 1,000 high-difficulty English queries drawn from the company's own Deep Research product; after rewriting, augmentation and filtering, 100 were sampled from the filtered pool in proportion to the product's mix of domains and reviewed by the company's domain experts, giving tasks that the authors describe as "anonymized, open-ended and complex, objectively evaluable, and representative", spanning 10 domains and drawing on information sources from 40 countries. Each task carries a rubric averaging roughly 40 binary criteria across four dimensions. The paper's primary judge was Gemini 3 Pro, with GPT-5.2 and Claude Sonnet 4.5 as secondary judges; each answer was graded in five independent runs and the standard deviation across runs is reported. The authors' Appendix A notes "systematic differences in absolute score levels" between judges (GPT-5.2 "consistently assigned lower scores"), and its Table 15 gives each of the seven systems' scores under all three judges: GPT-5.2 scored them 14.8 to 23.9 points below Gemini 3 Pro and 16.4 to 27.4 points below Sonnet 4.5, while those two judges differed by 0.2 to 12.0 points.

The paper's Table 8 reports, for example, the company's own Deep Research product (backed by Claude Opus 4.6) at $70.5 \pm 0.3\%$; the company's blog post on the benchmark [Perplexity AI, 2026b] is dated 4 February 2026, eight days before the paper. The post could not be retrieved by automated means when this appendix was first written; read on 26 October 2026, it reports four systems, and its figures for the company's product (pass rates of 89.4% on Law and 82.4% on Academic, latency 459.6 seconds) are those of the Opus 4.5 version in the paper's Tables 12 and 10, not of the Opus 4.6 version above. The benchmark paper's own tables are therefore cited for every figure, and the post is cited for completeness only.

A.2 Published scores

The Microsoft run [Anand, 2026] is a Microsoft Tech Community post of 30 March 2026 by Gaurav Anand; its statistics are independently quoted by Thomas [2026]. The post also gives per-dimension gains (+3.33 breadth and depth, +3.04 presentation, +2.58 factual accuracy); these are measured against Microsoft's own single-model Researcher under the same GPT-5.2 judge, not against Perplexity Deep Research. The Microsoft run is the only published DRACO-style run we found that reports a standard error, a significance test, and the number of judge repeats together. The OpenRouter post [OpenRouter, 2026] of 12 June 2026 reports no error bars and graded each response three times, where the paper used five grading runs; it states that its judge was Gemini 3.1 Pro Preview "instead of the paper's choice of Gemini 3 Pro", that the budget panel was synthesised by the same model as the main ensemble, and that "the paper reports 10–25 point shifts between judges". That range is not stated in the DRACO paper, but it roughly summarises the paper's Table 15, where GPT-5.2 scores each system 14.8 to 27.4 points below the other two judges; the range should therefore be read as the aggregator's summary of the authors' own figures, not as a separate

measurement. The Valyu post [Valyu, 2026] describes multi-step retrieval across specialised data sources and does not describe multi-model ensembling, states neither its judge model nor its run date, and reports no error bars, but does state that its datasets, run scripts, raw outputs and judge evaluations are published, which is the retention practice Section 7.4 requires; it is the only one of the four sources in Table 1 (the OpenRouter post gives two of its rows) to state this.

A.3 Third-party trackers

Two public leaderboards that aggregate DRACO results were consulted on 10 and again on 11 September 2026. One [LLM Stats, 2026] showed a top score of 0.772 (that is, 77.2%) "across 3 evaluated AI models", all three self-reported and none verified; the other [BenchLM, 2026] showed, on 10 September, 88.6% for the leading entry among three submitted system set-ups. A score of 88.6% does appear in a primary source not listed above: Anthropic's system card for Claude Opus 5 (24 July 2026, Figure 8.10.4.A) reports it for that model at its highest reasoning effort, graded by Claude Opus 4.6 as the mean of five grading runs [Anthropic, 2026c], which is above every score in Table 1. The 0.772 appears neither in the primary sources above nor in that system card, and the two trackers disagree with each other about which system leads. The first page showed 0.732 "across 1 evaluated AI model" when first consulted and 0.772 "across 3" a day later, which illustrates the point made in Section 3.

System	Type	Score	Judge	Precision reported
Valyu DeepResearch, "Heavy" mode [Valyu, 2026]	retrieval-centred product	72.7%	not stated	none; run date not stated
Perplexity Deep Research (Opus 4.6) [Zhong et al., 2026]	benchmark author's product	70.5%	Gemini 3 Pro	±0.3 SD over five runs
OpenRouter "Fusion" (Fable 5 + GPT-5.5, synthesised by Opus 4.8) [OpenRouter, 2026]	synthesis ensemble	69.0%	Gemini 3.1 Pro Preview	none; scored on 93 of 100 tasks, since Fable 5's content filters blocked the other 7
OpenRouter budget panel (three inexpensive models, synthesised by Opus 4.8) [OpenRouter, 2026]	synthesis ensemble	64.7%	Gemini 3.1 Pro Preview	none
Microsoft 365 Copilot Researcher "Critique" [Anand, 2026, Thomas, 2026]	multi-model research assistant	57.4%	GPT-5.2 ("the strictest of the three")	+7.0 points (SEM ±1.90) over Perplexity Deep Research's 50.4% under the same judge (the DRACO paper's Table 15); against its own single-model version, paired t-test $p < 0.0001$ on every dimension and significant in 8 of 10 domains; five runs per question

Table 1: Published DRACO scores of the systems discussed in Section 2.2, as of September 2026, in descending order. Each is what the DRACO paper calls a "normalized score", with the criteria weighted (its Table 8), not the unweighted pass rate of its Table 9. Scores under different judges are not comparable: the Microsoft figure was produced by the judge the benchmark's own authors identify as the strictest, the OpenRouter figures by a newer model than the paper's primary judge. The single-model scores in the OpenRouter post (among them Fable 5 65.3% on the 93 tasks it completed, GPT-5.5 60.0% and Opus 4.8 58.8%, and the budget panel's three members, DeepSeek V4 Pro 60.3%, Kimi K2.6 53.7% and Gemini 3 Flash 43.1%) are what the ensembles are compared against.

B A Worked Cost Instance at Current Commercial Terms

This appendix instantiates the algebra of Sections 4–7.2 with concrete values: the commercial terms are dated and sourced, while $\alpha = 20$, $\rho = 0.2$, the floor $n^* = 60$ and the pilot's σ are chosen for illustration.

B.1 Prompt-caching terms

As read on 11 September and again on 26 October 2026 (27 October where stated below), the three largest commercial API providers bill a repeated prompt prefix as follows. OpenAI charges cached input tokens at 0.1× the standard input price (0.05× for GPT-6.1 Sol and GPT-6.1 Terra, as of 26 October 2026), applies caching automatically to prefixes of at least 1,024 tokens, bills the first, cache-writing call at 1.25× for its default lifetime, and keeps a prefix eligible for reuse for at least 30 minutes [OpenAI, 2026a]. Anthropic charges cached reads at 0.1× (0.025× for Claude Fable 5.1 and Mythos 5.1, and, as of 26 October 2026, 0.05× for Claude Opus 5.5 and Sonnet 5.5), bills the first, cache-writing call at 1.25× for a five-minute cache lifetime, and requires minimum prefixes of 512–4,096 tokens depending on model [Anthropic, 2026a]. Google applies caching automatically on Gemini 2.5 and later models, with minimum prefixes of 2,048–4,096 tokens depending on model[2], charges cached reads at 0.1× on most models (0.05× on its flagship, Gemini 4 Argon, as of 26 October 2026) and adds a per-hour storage charge for explicitly created caches [Google, 2026]. None of the three caches output tokens. In the notation of Equation 5, the representative values are $c = 0.1$ and $w = 1$ (Google) or $w = 1.25$ (OpenAI, Anthropic). All three pages were re-read on 11 September 2026, and all three again on 26 October 2026, which is where OpenAI's cache-write charge and 30-minute minimum, and the three rates dated to that day, come from. OpenAI's terms apply to GPT-5.6 and later models, Anthropic bills a one-hour cache write at 2×, and Google's storage charge ranges from $0.50 to $1.80 per million tokens per hour for its Flash and Flash-Lite models and from $4.50 to $8.10 for its Pro models, depending on model and service tier.

B.2 The setting-cost ratio α

Artificial Analysis ran a fixed evaluation suite against one frontier model, GPT-5, at each of its four reasoning-effort settings. The suite consumed 3.5 million output tokens at the `minimal` setting and 82 million at `high`, a 23× difference, for Intelligence Index scores of 44 and 68 respectively [Artificial Analysis, 2025]. Because reasoning tokens are billed as output tokens at the same rate regardless of setting [OpenAI, 2026b, Anthropic, 2026d], this is a ratio of output costs of about 23 for that model on that suite, and an upper bound on its α there: the suite's inputs, about 100,000 tokens per question in its long-context benchmark, are paid at both settings and pull the cost ratio below the token ratio (Section C.3). The same service publishes the token counts and cost of running its whole suite for the models and settings it evaluates [Artificial Analysis, 2026]; α should be measured on the benchmark's own tasks for the systems under comparison, since the published figures hold for that suite only. Appendix C reports a measurement on analytic prompts, with α between 1.75 and 3.18 by output tokens (1.74 to 3.14 by cost) for the `high` setting against a lower one, and its consequence for the worked instance below.

B.3 Call counts for a modest configuration

For $C = 5$ systems, $T = 60$ tasks, $E = 2$ settings, $R = 3$ regenerations, $J = 3$ judges and $K = 5$ repeats, Equation 1 implies $C \cdot E \cdot R \cdot T = 1{,}800$ generation calls, Equation 2 implies 27,000

[2] `https://ai.google.dev/gemini-api/docs/caching`, read 27 October 2026.

unbatched judging calls, and Equation 3 implies 5,400 batched judging calls, before any per-call price is attached. The full protocol of the worked instance below, with $T_{\text{full}} = 180$ tasks, makes three times as many: 5,400 generation calls, 81,000 unbatched and 16,200 batched judging calls.

B.4 The batching factor β

With $C = 5$ and $\rho = 0.2$, Equation 8 gives $\beta = 3.0$ without caching ($c = w = 1$). With $c = 0.1$ and $w = 1$ it gives $\beta = 1.2$, and the ceiling $1 + (C - 1)\,c$ is 1.4; with $w = 1.25$, $\beta = 1.325$, on the assumption that the batched call itself pays no cache write, which holds where caching is requested call by call, as at Anthropic, but not where it is automatic, as at OpenAI (above). Where the batched call writes its prefix too, the denominator $1 + C\rho$ of Equation 8 becomes $w + C\rho$, and at OpenAI's $w = 1.25$, $\beta = 2.65/2.25 \approx 1.18$. Batching therefore saves a factor of 3 on judging cost without caching and, with it, a factor of 1.18 to 1.325, about a seventh to a quarter of the judging cost.

B.5 The combined reduction

Take $\alpha = 20$ with the cheap setting kept (S1), $R = 3$, $K = 5$, $E = 2$, $C = 5$, $\rho = 0.2$, and a full task count $T_{\text{full}} = 180$ reduced to an assumed floor $n^* = 60$. Then Equation 11 gives $F_G = 21 \cdot 3 \cdot 3 = 189$ and $F_J = 2 \cdot 3 \cdot 5 \cdot 3 \cdot \beta = 90\beta$, that is 270 with uncached batching and 108 under cached-read pricing with no write premium ($w = 1$, Google's terms); at Anthropic's $w = 1.25$, where the batched call pays no write, it is about 119, and the cached row of Table 2 becomes about 179, 161 and 146×; at OpenAI's, where the batched call pays the write too ($\beta \approx 1.18$, above), it is 106, and that row becomes about 175, 153 and 136×, within two of Google's. Table 2 evaluates Equation 12 across plausible values of the judging share s. The floor of 60 is assumed for illustration and does not follow from the power calculation below, whose inputs give $n^* = 128$ at $R = 3$ and $n^* = 184$ once S2 reduces R to 1, as it does here. On those inputs the floor exceeds $T_{\text{full}} = 180$, S6 saves nothing (the reduced protocol keeps all 180 tasks, a factor of 1, not 3, and even so reaches a power of only 0.79 at $R = 1$, and less once its $K = 1$ is counted), and because F_G and F_J both carry that factor, every reduction in the table falls to a third, 46–74×.

Judging share of full cost, s	0.1	0.3	0.5
Reduction, no prompt caching ($F_J = 270$)	195×	208×	222×
Reduction, cached-read pricing, $w = 1$ ($F_J = 108$)	176×	154×	137×

Table 2: Combined reduction from S1–S4 and S6 (Equation 12) for the worked instance, with $F_G = 189$. Multiplying every factor against the total cost would give $(1 + \alpha) \cdot E \cdot R \cdot K \cdot \beta \cdot T_{\text{full}}/n^* = 5{,}670\times$ for the same inputs without caching and 2,268× with it at $w = 1$ (2,504× at Anthropic's $w = 1.25$, 2,226× at OpenAI's), the overstatement Section 6 describes. The reference implementation reproduces every entry of this table (`tests/simplifications.test.mjs`).

B.6 The task floor n^*

For three pre-registered comparisons at family-wise $a = 0.05$, Holm's smallest per-comparison threshold is $a' = 0.05/3$; one-sided, $z_{a'} \approx 2.13$. At power $\pi = 0.8$, $z_\pi \approx 0.84$. With a pilot standard deviation of task-level paired differences $\sigma = 19$ points on a 100-point scale and a minimum difference in mean score worth detecting of $\delta = 5$ points, Equation 13 gives $n^* = 128$ tasks. By Equation 14, if that σ was measured at $R = 3$ with within-task variances $\sigma_A^2 = \sigma_B^2 = 120$ and

between-task variance $\omega^2 = 281$, then dropping to $R = 1$ (S2) raises σ^2 from 361 to 521 and n^* from 128 to 184, the interaction Section 7.2 describes. The reference implementation reproduces both figures (`tests/power.test.mjs`).

C Measurements Made for This Paper

The appendices above rely on published figures. Several premises of the cost model and of the simplifications are inexpensive enough to be measured directly, and three experiments measured them on 11 September 2026 with a small, pre-stated budget; the scripts, prompts and raw provider usage records are in the repository under `experiments/`; `experiments/MANIFEST.pdf` lists every run with its purpose, scripts, models, stated and actual spend, and result files with their SHA-256 hashes; its machine-readable copy, `experiments/RUNS.json`, also gives each run's caveats. Both date from September 2026 and are kept as the runs' own record; some of their headline figures (for E2 the prefix and prompt lengths, ρ and the ranking across orderings; for E3 the eight retried cells and $0.39) were later corrected against the result files, and the figures below replace them. Every figure below is a single small measurement made on one day and is reported so that a reader can judge the premise; the validation protocol of Section 8 is where such measurements belong at scale.

C.1 Judging-side premises (experiment E2)

Set-up. The first of the five E1 prompts (listed in Section C.3), a rubric of 40 binary criteria (about as many as a DRACO task has, but generated in the script from ten generic criteria, each repeated with four different qualifiers, and not written for the task), three short candidate answers of deliberately different quality, written in general terms rather than for the banking task (the one written to be strongest assumes a self-reported outcome, where the task has account-level default data), and OpenAI's `gpt-5.6-sol`, the first E1 model, as judge at its lowest reasoning setting, at the provider's default sampling (the model rejects a temperature parameter). The judge was told to output only its 40 verdicts and a final score line, with no justification before them, unlike DRACO's judges (S3). Total spend: $0.19 across 11 calls as the script records it, pricing every input token at the full rate; with the cached-read rate of 0.1× (Appendix B) on the 1,089 cached tokens of each of five calls it is about $0.16.

Prompt caching. Three consecutive calls shared the rubric prefix and differed only in the answer. The provider reported no cached tokens on any of the three: the shared prefix (a rubric of about 920 tokens, then the task and question) was about 1,005 tokens, just below the provider's documented 1,024-token minimum (Appendix B). Five later calls with an identical full prompt of 1,092 tokens each reported 1,089 of them as cached. Equation 5 therefore holds only once the fixed prefix clears the provider's minimum; a rubric of 40 short criteria, about 920 tokens here, does not do so on its own at this provider's 1,024-token minimum, though it would clear the 512-token minimum of some Anthropic models, and a protocol relying on the cached rate should measure its prefix length. The same 11 calls give the fixed and marginal token counts of Equation 8: about 1,005 fixed tokens and about 90 to 100 marginal input tokens per additional short answer on average (about 45 to 150 for the three answers, which differ in length), a ratio of about 0.1 in input tokens alone. Equation 8 defines ρ on cost, however, and scoring each answer also produced about 220 to 270 output tokens of verdicts, priced at 6× the input rate ($30 against $5 per million), so on these calls ρ is between about 1.4 and 1.7. Answers of the length deep-research tasks produce would raise ρ considerably. At $C = 5$, a ρ of 1.4 to 1.7 gives a batching factor β (Equation 8) of about 1.4 to 1.5 without caching and 1.04 to 1.08 with it, against 3.0 and 1.18 to 1.325 at the illustrative $\rho = 0.2$ of Appendix B.

Test–retest (S3). The same answer was scored five times with no memory between calls. The scores were 50, 50, 60, 50 and 43 on the 100-point scale: mean 50.6, standard deviation 6.1, range 17 points. These are the judge's own final score lines, which the rubric asks to equal the percentage of criteria marked `MET`; three of the five do not (21 of 40 marked `MET` was scored 50 twice, and 18 of 40 was scored 43), and the verdicts themselves give 52.5, 52.5, 60, 50 and 45, a range of 15 points. The first of the three caching calls above sent the identical prompt and scored the answer 53, inside that range; over all six scorings the mean is 51.0 and the standard deviation 5.5. A single scoring of this answer could have fallen anywhere within a 17-point interval, which is the quantity S3 says is lost at $K = 1$.

Order sensitivity (S4, Section 7.3). The three answers were scored together in one call, under explicit instructions to grade each independently against the absolute rubric, in three rotations of their order. The ranking of the three answers was the same in the first two orderings; in the third, the answer written to be strongest scored two points below the middle one, and the weakest stayed last. Scored alone, in the three caching calls above, the two came out as in the third ordering: 53 for the middle answer and 43 for the strongest. The answer written to be strongest scored 60, 65 and 48 depending on position, the middle one 45, 55 and 50, the weakest 25, 25 and 20. An absolute score moved by up to 17 points between orderings, the same range as the repeated scorings of one answer above, so with one call per ordering this run cannot separate the effect of order from the judge's test–retest noise. Section 2.2 notes that the choice of judge moves published DRACO scores; here scores moved between orderings within one judge, as they would under position bias, which is why Section 7.3 asks for the order to be varied across the K repeats.

C.2 The setting-cost ratio α (experiment E1)

Set-up. Five open-ended analytic prompts of a business-analysis kind (banking, mining, automotive, retail and telecommunications scenarios, each asking for a method and its most likely failure mode), one system instruction asking for a thorough professional answer, and two frontier models, one from OpenAI and one from Anthropic, each queried once per prompt at some of its documented reasoning-effort settings: `none`, `low` and `high` for the first, `low` and `high` for the second. Output was capped at 24,000 tokens; no call reached the cap. The calls for six cells failed with transient provider errors (ten failed calls in all) and were retried until they succeeded, so that every cell is filled. Total spend: $2.17 across 25 successful calls.

Result. Table 3 gives the paired ratios. For the first model the `high` setting consumed 2.83× the output tokens of `low` (per prompt 1.52–3.99) and 3.18× those of `none`; at `high` the provider reported between 1,024 and 4,068 reasoning tokens per answer, at `none` zero, and `low` consumed only 1.12× the output tokens of `none`. For the second model `high` consumed 1.75× the tokens of `low` (per prompt 1.48–2.00), with mean outputs of 3,600 tokens at `low` and 6,300 at `high`, thinking included. Cost ratios are slightly lower than the token ratios (2.80, 3.14 and 1.74 against 2.83, 3.18 and 1.75; both are 1.12 for `low` against `none`), because the same input cost is added at both settings. Mean latency rose from 38 seconds at `none` to 126 at `high` for the first model and from 58 to 97 seconds between `low` and `high` for the second.

Interpretation. The published 23× of Appendix B was measured on an evaluation suite of reasoning, knowledge and coding benchmarks; on analytic prose tasks the `high` setting of the two models measured here uses 1.75 to 3.18 times the output tokens of a lower one (1.74 to 3.14 times the cost). α appears to be a property not of a model alone but of a model on a task family under a given instruction (the first model gave 1.48 on the competition problems of E3, Section C.5, against 2.83

Provider	Settings compared	Paired prompts	α by tokens	Per-prompt range
first provider	high / low	5	2.83	1.52–3.99
first provider	high / none	5	3.18	2.04–4.25
first provider	low / none	5	1.12	0.94–1.34
second provider	high / low	5	1.75	1.48–2.00

Table 3: Measured setting-cost ratios on five analytic prompts, 11 September 2026. Pooled ratio of summed output tokens over the same five prompts; raw usage records in `experiments/results/`.

here; the second gave 2.67 there against 1.75 here, but only because of one problem, without which its ratio there is 1.68; E3 also asked for brief reasoning where E1 asked for a thorough answer, so the two differ in instruction as well as task family), and the pilot of Section 8 should measure it on the benchmark's own tasks before S1's saving is counted. The consequence for the worked instance of Appendix B is material: replacing its illustrative $\alpha = 20$ with the measured token ratios of the `high` setting against a lower one (1.75 to 3.18, Table 3) gives a generation-side factor F_G of 25 to 38 instead of 189, and a combined reduction (Equation 12) of roughly 27–66× across the same range of s and caching assumptions instead of 136–222×, about one and a half orders of magnitude instead of two. If the expensive setting is the one retained (Section 5, S1), the same configuration yields 13–27×. Both ranges are at the assumed floor $n^* = 60$; at the floor of 184 that the instance's own power inputs give, S6 saves nothing and they fall to a third, 9–22× and 4–9×. The framework's conclusion that the saving is dominated by the generation side and that judge count is the wrong place to cut is unchanged; the size of the overall saving is set by α and by the task floor, and both must come from the pilot's measurements.

C.3 Full method for experiment E1

This subsection gives the information required to reproduce E1 independently of the repository; the scripts (`experiments/e1-alpha.mjs`, `e1-retry.mjs`), the prompt file and the raw provider responses' usage blocks are also committed.

Models and interfaces. First provider: OpenAI, model `gpt-5.6-sol`, endpoint `POST https://api.openai.com/v1/chat/completions`, request fields `model`, `messages` (one `system` and one `user` message), `max_completion_tokens` = `24000`, `reasoning_effort` ∈ {`none`, `low`, `high`}; no temperature field (the model rejects it). Second provider: Anthropic, model `claude-opus-5`, endpoint `POST https://api.anthropic.com/v1/messages`, header `anthropic-version` = `2023-06-01`, fields `model`, `max_tokens` = `24000`, `system`, `messages` (one `user` message), `output_config.effort` ∈ {`low`, `high`}; no thinking budget set explicitly and no temperature field. Model identifiers and prices ($5 input and $30 output per million tokens for the first; $5 and $25 for the second) are as the providers listed them on 11 September 2026.

System instruction (verbatim). "You are a careful analyst. Answer the question thoroughly for a professional reader: state your method, the key assumptions, the most likely failure modes, and what evidence would change your conclusion. Use headings and be concrete."

User message. The prompt's context, a blank line, then "Question: " followed by the question. The five prompts, verbatim:

1. **banking** We want to know whether a collections strategy introduced in March reduced 90-day default rates. Accounts were assigned by branch, not randomly, and the branches that adopted

early had the worst arrears to begin with. We have 30 months of monthly account-level data. *Question:* What method would you use, and what is the most likely way this analysis produces a confidently wrong answer?

2. **mining** We want to know whether a new blast design improved fragmentation. It was trialled on 30 of 200 blasts, chosen by the drill and blast engineers in areas they judged suitable. Fragmentation is measured by image analysis on the muckpile, which is affected by how the shovel digs. *Question:* How would you estimate the effect, and what is the most likely way this analysis misleads?
3. **automotive** Warranty claims for a component are rising. Claims are recorded at dealer level with free-text fault descriptions, build dates come from a separate system, and a supplier change happened partway through the model year. Vehicles built later have had less time in service. *Question:* How would you establish whether the supplier change caused it, and what is the most likely false conclusion?
4. **retail** We have 24 months of weekly units sold by store and SKU, about 6 million rows. A price cut was applied to one category in week 60, but only in stores above a footfall threshold. Some stores closed for refurbishment during the period. *Question:* What method would you use, and what is the most likely way this analysis produces a confidently wrong answer?
5. **telecoms** We want to know whether a network upgrade reduced churn. It rolled out cell by cell over a year, prioritising the worst-performing cells. Customers are assigned to a home cell by billing address, which is often not where they actually use the service. *Question:* How would you estimate the effect, and what is the most likely way this analysis misleads?

Procedure. Calls were made sequentially in the order model × effort × prompt, one call per cell, on 11 September 2026 between 07:41 and 08:20 UTC (the first run) and between 08:55 and 09:10 UTC (the retries). Ten calls in six cells, six in the first run and four among the retries, failed with the provider's HTTP 500 `server_error` and were retried, up to three times with a delay, until they succeeded; the retried cells are marked in Table 4. No call was excluded. No call reached the output cap (all finish reasons `stop` or `end_turn`).

Quantities counted. Output tokens are the provider's own `usage` fields: `completion_tokens` (first provider; this includes the reasoning tokens, which the provider also reports separately as `completion_tokens_details.reasoning_tokens`) and `output_tokens` (second provider; this also includes the thinking tokens, which the provider reports separately as `output_tokens_details.thinking_tokens`). Input tokens are `prompt_tokens` and `input_tokens`. Cost is input tokens times the input price plus output tokens times the output price, at the prices above; no cached-input discount applied (every prompt, system instruction included, came to 125 to 199 input tokens, far below the caching minimum, and every usage record reports zero cached tokens). Latency is wall-clock time of the HTTP request.

Analysis. For each model and pair of settings, α, which S1 defines on cost, is estimated by the ratio of the *sums* of output tokens over the five prompts (pooled ratio); the per-prompt ratios are reported alongside as the range. The cost ratio is computed the same way; it is slightly lower than the token ratio, by at most 0.04, because the identical input cost is added to both sums. No statistical test is applied: with five prompts the result is a point estimate with a stated range, offered as evidence that α is not a constant of the effort setting, and not as an estimate of α for any task population.

Comparison with the published 23×. The published figure [Artificial Analysis, 2025] is not from the DRACO paper, which does not measure α; it is the ratio of the total output tokens used to run an evaluation suite at two settings. The two measurements differ in every design dimension, so their disagreement is not a contradiction but an open question, taken up in Section C.4:

	Published (Artificial Analysis)	This paper (E1)
Model	GPT-5 (2025)	gpt-5.6-sol; claude-opus-5 (2026)
Settings	`minimal` vs `high`, the lowest and highest of four	`none`/`low` vs `high`; `low` vs `high`; `xhigh` and `max`, above `high` for both models, not run
Tasks	the Intelligence Index suite (reasoning, knowledge and coding benchmarks; eight in August 2025, listed in Section C.4)	five open-ended analytic prose prompts
Tools / browsing	per suite	none
Prompts	thousands (suite total)	five
Repeats	per suite	one call per cell
Token accounting	suite-total output tokens	provider usage fields per call
Instruction	benchmark harness prompts	one system instruction asking for a thorough answer with headings
Result	23× (3.5M → 82M)	2.83× (3.18× vs `none`); 1.75×

C.4 Why the two measurements of α differ

Four candidate explanations for the distance between the published 23× and the measured 1.75–3.18× are set out below; each would let both figures stand, and together they make α a quantity that must be measured on the tasks of interest. Experiment E3 (Section C.5) tests the first of them and does not support it.

Task family. The published figure was measured on an evaluation suite composed of "MMLU-Pro, GPQA Diamond, Humanity's Last Exam, LiveCodeBench, SciCode, AIME, IFBench & AA-LCR" [Artificial Analysis, 2025], most of which are competition-, examination- or code-style problems with a short, checkable answer. On such problems most of the output at a high setting is reasoning (in E3, Section C.5, 72% for the first model and 60% for the second, against 69% and 39% on the E1 prompts), and the literature on reasoning length shows the amount of reasoning a model spends varies by problem type and between models, and suggests that it changes across model generations: Ballon et al. [2026] find that accuracy declines as reasoning chains grow even at fixed difficulty, though less so in more proficient models, which they take to suggest that "new generations of reasoning models use test-time compute more effectively"; Lee et al. [2025] show that "there exists an optimal amount of reasoning tokens such that the probability of producing a correct solution is maximized" and that it grows with the length of the underlying computation a task requires; and Du et al. [2025] report that "models solving the same problem with similar accuracy can exhibit up to a 5.0× difference in token length". Our prompts asked for a structured analysis in prose, where the visible answer is itself long at every setting: about 930 to 1,390 tokens per answer for the first model and 2,970 to 4,040 for the second, reasoning excluded.

Arithmetic of the ratio. Because α is a ratio of whole-call costs, estimated here by the ratio of *total* output tokens, a large visible answer compresses it. In E1 the first model produced about 1,150 tokens at `none` with zero reasoning tokens and about 3,660 at `high`, of which about 2,520 were reasoning (1,024 to 4,068 per answer): measured on reasoning tokens alone the ratio is unbounded, measured on total tokens it is 3.18. A benchmark whose answers are a single number has almost no visible output, so the same reasoning behaviour yields a far larger total-token ratio. This follows from the definition and says nothing about the models.

Effort semantics. Both providers now document adaptive behaviour that narrows the gap between settings on tasks that do not need long reasoning. OpenAI's guide states that its models "reason

Model	Effort	Prompt	In	Out	Reasoning	Latency (s)	Stop
claude-opus-5	high	1 banking	191	5,704	1,717	89	end_turn
claude-opus-5	high	2 mining	199	6,641	2,967	106	end_turn
claude-opus-5	high	3 automotive	184	6,702	3,093	102	end_turn
claude-opus-5	high	4 retail	183	6,071	2,036	91	end_turn
claude-opus-5	high	5 telecoms	180	6,298	2,395	97	end_turn
claude-opus-5	low	1 banking	191	3,370	326	54	end_turn
claude-opus-5	low	2 mining	199	3,543	263	57	end_turn
claude-opus-5	low	3 automotive	184	3,358	320	55	end_turn
claude-opus-5	low	4 retail	183	3,413	446	54	end_turn
claude-opus-5	low	5 telecoms	180	4,262	914	68	end_turn
gpt-5.6-sol	high	1 banking	129	2,464	1,536	138	stop (retry 1)
gpt-5.6-sol	high	2 mining	133	4,062	3,072	126	stop (retry 3)
gpt-5.6-sol	high	3 automotive	125	2,251	1,024	73	stop
gpt-5.6-sol	high	4 retail	128	5,203	4,068	160	stop
gpt-5.6-sol	high	5 telecoms	125	4,306	2,920	130	stop
gpt-5.6-sol	low	1 banking	129	1,343	341	43	stop
gpt-5.6-sol	low	2 mining	133	1,166	139	41	stop (retry 1)
gpt-5.6-sol	low	3 automotive	125	1,485	352	48	stop
gpt-5.6-sol	low	4 retail	128	1,303	325	43	stop
gpt-5.6-sol	low	5 telecoms	125	1,164	100	41	stop (retry 1)
gpt-5.6-sol	none	1 banking	129	1,105	0	39	stop (retry 2)
gpt-5.6-sol	none	2 mining	133	1,080	0	39	stop (retry 2)
gpt-5.6-sol	none	3 automotive	125	1,105	0	34	stop
gpt-5.6-sol	none	4 retail	128	1,225	0	37	stop
gpt-5.6-sol	none	5 telecoms	125	1,234	0	39	stop

Table 4: Every successful call of experiment E1 (25 calls, one per cell, 11 September 2026); the ten failed calls are not listed. In/Out/Reasoning are token counts from the providers' own usage fields; "Reasoning" is included in "Out" for both providers (`reasoning_tokens` for `gpt-5.6-sol`, `thinking_tokens` for `claude-opus-5`). Stop = provider finish reason, and "retry n" marks a cell whose call succeeded on its nth retry, after n failed calls; no call stopped on the output cap.

adaptively across reasoning efforts, using fewer tokens for simpler tasks and thinking harder for complex tasks" [OpenAI, 2026b]. Anthropic's states that "effort is a behavioral signal, not a strict token budget", that "at lower effort levels, Claude still thinks on sufficiently difficult problems, but thinks less", and, for the model measured here, that "effort controls thinking volume, not visible response length: on Claude Opus 5, changing effort does not reliably shorten responses" [Anthropic, 2026b]. A setting that behaves this way is unlikely to produce a 23× spread on prose tasks; the measured 1.75× fits that description: between the two settings the second model's thinking tokens rose 5.38× and its visible output only 1.23×.

Model generation and the published suite itself. The 23× was measured on the first model of its series in 2025; the models measured here are a later generation, and Artificial Analysis, which publishes the suite, has since changed the suite's composition, weighting and evaluation mechanics [Artificial Analysis, 2026], so the published number is not one that its own publisher would reproduce today on the same terms.

What would discriminate these. Two cheap measurements: the five E1 prompts on the 2025 model at `minimal` and `high`, which isolates model generation; and a set of competition-style problems through the E1 models at `low` and `high`, which isolates task family. The second is reported below as experiment E3.

C.5 Competition problems as a task-family test (experiment E3)

Set-up. The same two models and the `low` and `high` settings of E1, applied to the first five problems of the 2025 American Invitational Mathematics Examination (AIME, paper I), taken from the public dataset `math-ai/aime25` through the Hugging Face datasets API; system instruction: "Solve the competition mathematics problem. Show your reasoning briefly, then give the final answer as an integer on the last line in the form 'ANSWER: <integer>'." Output cap 16,000 tokens, never reached. One call per cell; the calls for six cells, all to the first provider, failed with transient HTTP 500 errors during an outage (three attempts each) and were retried until they succeeded. Total spend: $0.38 across 20 successful calls. The scripts (`e3-aime.mjs`, `e3-retry.mjs`), the problems and the raw usage records are committed alongside E1's.

Result. If task family explained the gap between the published 23× and E1, α should be far larger here than on prose. It is not (Table 5). For the first model the pooled ratio is 1.48 (median per problem 1.18; reasoning tokens 105–1,435 at `high` against 114–526 at `low`), *lower* than its 2.83 on prose; all ten answers were correct. For the second model the pooled ratio is 2.67 but heavy-tailed: the per-problem ratios are 0.97, 1.22, 1.76, 2.12 and 72.7, the last from a problem answered correctly in 26 tokens at `low` and in 1,890 at `high`; the median is 1.76, close to its pooled prose value of 1.75; accuracy was four of five at `low` and five of five at `high`.

Model	Setting	Out tokens	Pooled α	Median α	Correct	Per-problem α
gpt-5.6-sol	low	2,675			5/5	
gpt-5.6-sol	high	3,959	1.48	1.18	5/5	1.01, 2.02, 1.18, 1.11, 1.33
claude-opus-5	low	1,857			4/5	
claude-opus-5	high	4,960	2.67	1.76	5/5	1.76, 72.7, 0.97, 1.22, 2.12

Table 5: Experiment E3, 11 September 2026: the E1 models at `low` and `high` on AIME 2025 I problems 1–5. Out tokens are summed over the five problems, and pooled α is the ratio of those sums (by cost it is 1.46 for `gpt-5.6-sol` and 2.53 for `claude-opus-5`); per-problem values are in problem order.

Interpretation. E3 does not support task family as the explanation for these models on these items, and it exposes a confound that any such test must reckon with: a competition problem that a frontier model answers correctly in 26 tokens is not, for that model, a hard problem, and a public 2025 problem set is likely to have been seen in the training of models released in 2026. What E3 does show is consistent with the effort-semantics and model-generation explanations of Section C.4: for these models the `high` setting adds little on most items of E3, and the pooled ratio of `high` to `low` stays between about one and a half and three on both task families (1.48 to 2.83), with a heavy tail on individual items. The large additions did not come where `low` fell short: the three largest per-problem ratios (the second model's 72.7 and 2.12, the first model's 2.02) are on problems already answered correctly at `low`. On the only problem answered wrongly at `low` (problem 3, second model) `high` used slightly fewer tokens in all (358 against 370), though its thinking rose from 9 to 218 tokens, and answered it correctly. The published 23× may therefore be a property of a particular 2025 model, whose effort levels may have behaved as coarse switches (E3 used later models and cannot show this), on a suite that was hard for it at the time, and not a constant of the effort setting. A test of task family that survives this confound would need problems demonstrably unseen by the models and hard enough that `low` fails on them, which the pilot of Section 8 can supply, where the benchmark's tasks have not been published, from its own retained data.

References

AllChat. AllChat, 2026. URL https://askallchat.com/. Product site; its benchmarks page is https://askallchat.com/benchmarks. Accessed 10 September 2026; both pages re-read 26 October 2026.

Gaurav Anand. Introducing multi-model intelligence in Researcher. Microsoft Copilot Blog, Microsoft Tech Community, March 2026. URL https://techcommunity.microsoft.com/blog/microsoft-copilot-blog/introducing-multi-model-intelligence-in-researcher/4506011. Published 30 March 2026. Accessed 10 September 2026; re-checked at its new address 25 October 2026.

Anthropic. Prompt caching. Claude Developer Platform documentation, 2026a. URL https://platform.claude.com/docs/en/build-with-claude/prompt-caching. Cache reads 0.1× base input price (0.025× for Claude Fable 5.1 and Mythos 5.1); cache writes 1.25× (5-minute) or 2× (1-hour); minimum cacheable prompt 512–4,096 tokens by model; output tokens not cached. Accessed 10 September 2026 and re-read 11 September 2026. Re-read 26 October 2026: cache reads 0.05× for Claude Opus 5.5 and Sonnet 5.5.

Anthropic. Effort. Claude Developer Platform documentation, 2026b. URL https://platform.claude.com/docs/en/build-with-claude/effort. Accessed 11 September 2026.

Anthropic. Claude Opus 5 system card. Anthropic, July 2026c. URL https://www-cdn.anthropic.com/c5fbac3f0b1280a933ebd26d3cb8bb9f5bdeaf48/Claude%20Opus%205%20System%20Card.pdf. Dated 24 July 2026; DRACO in Section 8.10.4 and Figure 8.10.4.A (Claude Opus 4.6 as judge, mean of five grading runs). Accessed 26 October 2026.

Anthropic. Extended thinking. Claude Developer Platform documentation, 2026d. URL https://platform.claude.com/docs/en/build-with-claude/extended-thinking. Thinking

tokens are billed as output tokens; `budget_tokens` minimum 1,024; on newer models reasoning depth is set by an `effort` level. Accessed 10 September 2026.

Artificial Analysis. GPT-5 benchmarks and analysis, August 2025. URL https://artificialanalysis.ai/articles/gpt-5-benchmarks-and-analysis. Published 7 August 2025. Intelligence Index 68 / 67 / 64 / 44 at high / medium / low / minimal reasoning effort; 82M output tokens at high versus 3.5M at minimal to run the suite. Accessed 10 September 2026.

Artificial Analysis. Intelligence benchmarking methodology, 2026. URL https://artificialanalysis.ai/methodology/intelligence-benchmarking. Accessed 10 September 2026; re-read 26 and 27 October 2026.

Berk Atil, Sarp Aykent, Alexa Chittams, Lisheng Fu, Rebecca J. Passonneau, Evan Radcliffe, Guru Rajan Rajagopal, Adam Sloan, Tomasz Tudrej, Ferhan Ture, Zhe Wu, Lixinyu Xu, and Breck Baldwin. Non-determinism of "deterministic" LLM settings, August 2024. URL https://arxiv.org/abs/2408.04667. v5, 2 April 2025.

R. Harald Baayen, Douglas J. Davidson, and Douglas M. Bates. Mixed-effects modeling with crossed random effects for subjects and items. *Journal of Memory and Language*, 59(4):390–412, 2008. doi: 10.1016/j.jml.2007.12.005. URL https://doi.org/10.1016/j.jml.2007.12.005.

Marthe Ballon, Andres Algaba, and Vincent Ginis. The relationship between reasoning and performance in large language models – o3 (mini) thinks harder, not longer. *Scientific Reports*, 16:20896, 2026. doi: 10.1038/s41598-026-50923-2. URL https://arxiv.org/abs/2502.15631. arXiv v1, 21 February 2025; v2, 7 July 2026.

BenchLM. DRACO benchmark leaderboard. benchlm.ai, 2026. URL https://benchlm.ai/benchmarks/draco. Accessed 10 and 11 September 2026; 3 entries on 10 September.

Samuel R. Bowman and George Dahl. What will it take to fix benchmarking in natural language understanding? In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4843–4855, June 2021. doi: 10.18653/v1/2021.naacl-main.385. URL https://aclanthology.org/2021.naacl-main.385/.

Sam Bowyer, Laurence Aitchison, and Desi R. Ivanova. Position: Don't use the CLT in LLM evals with fewer than a few hundred datapoints. In *Proceedings of the 42nd International Conference on Machine Learning (ICML 2025), position paper track*, volume 267 of *Proceedings of Machine Learning Research*, pages 81143–81184, 2025. URL https://arxiv.org/abs/2503.01747.

Robert L. Brennan. *Generalizability Theory*. Springer, New York, 2001. doi: 10.1007/978-1-4757-3456-0. URL https://doi.org/10.1007/978-1-4757-3456-0.

Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling, July 2024. URL https://arxiv.org/abs/2407.21787. v3, 30 December 2024.

Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why do multi-agent LLM systems fail? In *Advances in*

Neural Information Processing Systems 38 (NeurIPS 2025), Datasets and Benchmarks Track, pages 135676–135729, 2025. URL https://openreview.net/forum?id=MqBzKkb8eK. arXiv v3, 26 October 2025: https://arxiv.org/abs/2503.13657.

Lingjiao Chen, Jared Quincy Davis, Boris Hanin, Peter Bailis, Ion Stoica, Matei Zaharia, and James Zou. Are more LLM calls all you need? Towards scaling laws of compound inference systems, March 2024. URL https://arxiv.org/abs/2403.02419. v2, 4 June 2024.

Zhoujun Cheng, Jungo Kasai, and Tao Yu. Batch prompting: Efficient inference with large language model APIs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 792–810, December 2023. doi: 10.18653/v1/2023.emnlp-industry.74. URL https://aclanthology.org/2023.emnlp-industry.74/.

Wei-Lin Chiang, Lianmin Zheng, Ying Sheng, Anastasios Nikolas Angelopoulos, Tianle Li, Dacheng Li, Hao Zhang, Banghua Zhu, Michael Jordan, Joseph E. Gonzalez, and Ion Stoica. Chatbot Arena: An open platform for evaluating LLMs by human preference, March 2024. URL https://arxiv.org/abs/2403.04132.

Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. In *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)*, volume 235 of *Proceedings of Machine Learning Research*, pages 11733–11763, 2024. URL https://arxiv.org/abs/2305.14325.

Zheng Du, Hao Kang, Song Han, Tushar Krishna, and Ligeng Zhu. OckBench: Measuring the efficiency of LLM reasoning, November 2025. URL https://arxiv.org/abs/2511.05722. v3, 3 June 2026.

Yann Dubois, Balázs Galambosi, Percy Liang, and Tatsunori B. Hashimoto. Length-controlled AlpacaEval: A simple way to debias automatic evaluators. In *First Conference on Language Modeling (COLM 2024)*, 2024. URL https://arxiv.org/abs/2404.04475.

Michael D. Ernst. Permutation methods: A basis for exact inference. *Statistical Science*, 19(4): 676–685, 2004. doi: 10.1214/088342304000000396. URL https://doi.org/10.1214/088342304000000396.

Scott Frohn. The impact of LLM self-consistency and reasoning effort on automated scoring accuracy and cost, April 2026. URL https://arxiv.org/abs/2604.26954. Submitted 3 April 2026.

Google. Gemini developer API pricing. Google AI for Developers, 2026. URL https://ai.google.dev/gemini-api/docs/pricing. For Gemini 3.5–3.8 Flash: cached input priced at 0.1× standard input (e.g. $0.075 vs $0.75 per million tokens), plus cache storage per million tokens per hour. Accessed 10 September 2026 and re-read 11 September 2026. Re-read 26 October 2026: cached input 0.05× on Gemini 4 Argon.

Rodrigo Guedes de Souza and Alison R. Panisson. Who thinks best depends on how long you let them: Budget-dependent rankings in LLM evaluation, August 2026. URL https://arxiv.org/abs/2608.12150. Submitted 12 August 2026.

Horace He and Thinking Machines Lab. Defeating nondeterminism in LLM inference. Thinking Machines Lab, Connectionism, September 2025. URL https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/. Published 10 September 2025. Accessed 10 September 2026; re-read 25 October 2026.

Valentin Hofmann, David Heineman, Ian Magnusson, Kyle Lo, Jesse Dodge, Maarten Sap, Pang Wei Koh, Chun Wang, Hannaneh Hajishirzi, and Noah A. Smith. Fluid language model benchmarking. In *Second Conference on Language Modeling (COLM 2025)*, 2025. URL https://arxiv.org/abs/2509.11106.

Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979. URL https://www.jstor.org/stable/4615733.

Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. RULER: What's the real context size of your long-context language models? In *First Conference on Language Modeling (COLM 2024)*, 2024. URL https://arxiv.org/abs/2404.06654.

Dongfu Jiang, Xiang Ren, and Bill Yuchen Lin. LLM-Blender: Ensembling large language models with pairwise ranking and generative fusion. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 14165–14178, July 2023. doi: 10.18653/v1/2023.acl-long.792. URL https://aclanthology.org/2023.acl-long.792/.

Dong Kyun Kim and Joon-Yong Jung. Test-time compute and budget ("reasoning effort"): An underrecognized methodologic variable in medical large language model research. *Korean Journal of Radiology*, 27(8):780–784, 2026. doi: 10.3348/kjr.2026.0643. URL https://pmc.ncbi.nlm.nih.gov/articles/PMC13437210/.

Elliot Kim, Avi Garg, Kenny Peng, and Nikhil Garg. Correlated errors in large language models. In *Proceedings of the 42nd International Conference on Machine Learning (ICML 2025)*, volume 267 of *Proceedings of Machine Learning Research*, pages 30038–30066, 2025. URL https://arxiv.org/abs/2506.07962.

Seungone Kim, Juyoung Suk, Shayne Longpre, Bill Yuchen Lin, Jamin Shin, Sean Welleck, Graham Neubig, Moontae Lee, Kyungjae Lee, and Minjoon Seo. Prometheus 2: An open source language model specialized in evaluating other language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 4334–4353, November 2024. doi: 10.18653/v1/2024.emnlp-main.248. URL https://aclanthology.org/2024.emnlp-main.248/.

Guneet Kohli. Nine judges, two effective votes: Correlated errors undermine LLM evaluation panels, May 2026. URL https://arxiv.org/abs/2605.29800. Submitted 28 May 2026.

Daniël Lakens. Calculating and reporting effect sizes to facilitate cumulative science: A practical primer for t-tests and ANOVAs. *Frontiers in Psychology*, 4:863, 2013. doi: 10.3389/fpsyg.2013.00863. URL https://doi.org/10.3389/fpsyg.2013.00863.

Celine Lee, Alexander M. Rush, and Keyon Vafa. Critical thinking: Which kinds of complexity govern optimal reasoning length? In *Proceedings of the 14th International Joint Conference on Natural Language Processing and the 4th Conference of the Asia-Pacific Chapter of the*

Association for Computational Linguistics (IJCNLP-AACL 2025), pages 1038–1060, December 2025. doi: 10.18653/v1/2025.ijcnlp-long.57. URL https://aclanthology.org/2025.ijcnlp-long.57/.

Dawei Li, Bohan Jiang, Liangjie Huang, Alimohammad Beigi, Chengshuai Zhao, Zhen Tan, Amrita Bhattacharjee, Yuxuan Jiang, Canyu Chen, Tianhao Wu, Kai Shu, Lu Cheng, and Huan Liu. From generation to judgment: Opportunities and challenges of LLM-as-a-judge. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 2757–2791, November 2025. doi: 10.18653/v1/2025.emnlp-main.138. URL https://aclanthology.org/2025.emnlp-main.138/.

Haitao Li, Qian Dong, Junjie Chen, Huixue Su, Yujia Zhou, Qingyao Ai, Ziyi Ye, and Yiqun Liu. LLMs-as-judges: A comprehensive survey on LLM-based evaluation methods, December 2024a. URL https://arxiv.org/abs/2412.05579. v2, 10 December 2024.

Tianle Li, Wei-Lin Chiang, Evan Frick, Lisa Dunlap, Tianhao Wu, Banghua Zhu, Joseph E. Gonzalez, and Ion Stoica. From crowdsourced data to high-quality benchmarks: Arena-Hard and BenchBuilder pipeline, June 2024b. URL https://arxiv.org/abs/2406.11939. v2, 14 October 2024.

Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, et al. Holistic evaluation of language models. *Transactions on Machine Learning Research*, 2023. URL https://arxiv.org/abs/2211.09110.

Tian Liang, Zhiwei He, Wenxiang Jiao, Xing Wang, Yan Wang, Rui Wang, Yujiu Yang, Shuming Shi, and Zhaopeng Tu. Encouraging divergent thinking in large language models through multi-agent debate. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 17889–17904, November 2024. doi: 10.18653/v1/2024.emnlp-main.992. URL https://aclanthology.org/2024.emnlp-main.992/.

Bill Yuchen Lin, Yuntian Deng, Khyathi Chandu, Faeze Brahman, Abhilasha Ravichander, Valentina Pyatkin, Nouha Dziri, Ronan Le Bras, and Yejin Choi. WildBench: Benchmarking LLMs with challenging tasks from real users in the wild, June 2024. URL https://arxiv.org/abs/2406.04770. v2, 5 October 2024.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024a. doi: 10.1162/tacl_a_00638. URL https://aclanthology.org/2024.tacl-1.9/.

Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. G-Eval: NLG evaluation using GPT-4 with better human alignment. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 2511–2522, December 2023. doi: 10.18653/v1/2023.emnlp-main.153. URL https://aclanthology.org/2023.emnlp-main.153/.

Yinhong Liu, Han Zhou, Zhijiang Guo, Ehsan Shareghi, Ivan Vulić, Anna Korhonen, and Nigel Collier. Aligning with human judgement: The role of pairwise preference in large language model evaluators. In *First Conference on Language Modeling (COLM 2024)*, 2024b. URL https://arxiv.org/abs/2403.16950.

Adian Liusie, Potsawee Manakul, and Mark Gales. LLM comparative assessment: Zero-shot NLG evaluation through pairwise comparisons using large language models. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 139–151, March 2024. doi: 10.18653/v1/2024.eacl-long.8. URL https://aclanthology.org/2024.eacl-long.8/.

LLM Stats. DRACO benchmark leaderboard. llm-stats.com, 2026. URL https://llm-stats.com/benchmarks/draco. Accessed 10 and 11 September 2026; 3 self-reported entries, 0 verified, on 11 September.

Lovish Madaan, Aaditya K. Singh, Rylan Schaeffer, Andrew Poulton, Sanmi Koyejo, Pontus Stenetorp, Sharan Narang, and Dieuwke Hupkes. Quantifying variance in evaluation benchmarks, June 2024. URL https://arxiv.org/abs/2406.10229.

Felipe Maia Polo, Lucas Weber, Leshem Choshen, Yuekai Sun, Gongjun Xu, and Mikhail Yurochkin. tinyBenchmarks: Evaluating LLMs with fewer examples. In *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)*, volume 235 of *Proceedings of Machine Learning Research*, pages 34303–34326, 2024. URL https://proceedings.mlr.press/v235/maia-polo24a.html.

Martian. Introducing Martian – better AI tools through better understanding, July 2023. URL https://withmartian.com/post/introducing-martian---better-ai-tools-through-better-understanding. Published 20 July 2023. Accessed 10 September 2026.

Evan Miller. Adding error bars to evals: A statistical approach to language model evaluations, November 2024. URL https://arxiv.org/abs/2411.00640.

Brian A. Nosek, Charles R. Ebersole, Alexander C. DeHaven, and David T. Mellor. The preregistration revolution. *Proceedings of the National Academy of Sciences*, 115(11):2600–2606, 2018. doi: 10.1073/pnas.1708274114. URL https://pmc.ncbi.nlm.nih.gov/articles/PMC5856500/.

Not Diamond. Launching Not Diamond, July 2024. URL https://www.notdiamond.ai/blog/launching-not-diamond. Published 30 July 2024. Accessed 10 September 2026.

OpenAI. Prompt caching. OpenAI API documentation, 2026a. URL https://developers.openai.com/api/docs/guides/prompt-caching. Cached input tokens billed at 0.1× the uncached rate (GPT-5.6 and later); minimum cacheable prefix 1,024 tokens; automatic; output tokens not cached. Accessed 10 September 2026 and re-read 11 September 2026. Re-read 26 October 2026: cached input 0.05× for GPT-6.1 Sol and GPT-6.1 Terra; cache writes billed at 1.25× the uncached input rate for the default lifetime, which is a minimum of 30 minutes.

OpenAI. Reasoning models. OpenAI API documentation, 2026b. URL https://developers.openai.com/api/docs/guides/reasoning. "Reasoning tokens . . . are billed as output tokens"; `reasoning_effort` values none, minimal, low, medium, high, xhigh, max (model-dependent). Accessed 10 September 2026.

OpenRouter. Surpassing frontier performance with Fusion, June 2026. URL https://openrouter.ai/blog/announcements/fusion-beats-frontier/. Published 12 June 2026 (byline Brian Thomas), updated 13 July 2026. Accessed 10 September 2026.

Shuyin Ouyang, Jie M. Zhang, Mark Harman, and Meng Wang. An empirical study of the non-determinism of ChatGPT in code generation, August 2023. URL https://arxiv.org/abs/2308.02828. v2, 17 October 2024; published in ACM Transactions on Software Engineering and Methodology 34(2), 2025, pages 1–28, https://doi.org/10.1145/3697010.

Yotam Perlitz, Elron Bandel, Ariel Gera, Ofir Arviv, Liat Ein-Dor, Eyal Shnarch, Noam Slonim, Michal Shmueli-Scheuer, and Leshem Choshen. Efficient benchmarking (of language models). In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 2519–2536, June 2024a. doi: 10.18653/v1/2024.naacl-long.139. URL https://aclanthology.org/2024.naacl-long.139/.

Yotam Perlitz, Ariel Gera, Ofir Arviv, Asaf Yehudai, Elron Bandel, Eyal Shnarch, Michal Shmueli-Scheuer, and Leshem Choshen. Do these LLM benchmarks agree? Fixing benchmark evaluation with BenchBench, July 2024b. URL https://arxiv.org/abs/2407.13696. v2, 12 September 2024.

Perplexity AI. DRACO dataset. Hugging Face dataset card, 2026a. URL https://huggingface.co/datasets/perplexity-ai/draco. MIT licence. Accessed 10 September 2026.

Perplexity AI. Evaluating deep research performance in the wild with the DRACO benchmark. Perplexity blog, February 2026b. URL https://www.perplexity.ai/hub/blog/evaluating-deep-research-performance-in-the-wild-with-the-draco-benchmark. Vendor announcement, published 4 February 2026 (byline Perplexity Research). Accessed 26 October 2026; per-system scores are taken from Zhong et al. [2026], Table 8, not from this post.

William Philipp, Finn Fassbender, Daniel Fister, Thorsten Langer, Martje G. Pauly, Rebecca Herzog, Markus A. Hobert, Theresa Paulus, Alexander Baumann, Chi Wang Ip, Lukas L. Goede, Johanna Reimer, Sebastian Löns, Ronald Böck, and Sebastian Fudickar. Clinician-level agreement without clinical caution: LLM evaluator limits in medical AI benchmarking, July 2026. URL https://arxiv.org/abs/2607.01103v2. v2, 31 July 2026; v3 of 6 October 2026 is retitled and its abstract no longer carries the passages quoted here.

Zhen Qin, Rolf Jagerman, Kai Hui, Honglei Zhuang, Junru Wu, Le Yan, Jiaming Shen, Tianqi Liu, Jialu Liu, Donald Metzler, Xuanhui Wang, and Michael Bendersky. Large language models are effective text rankers with pairwise ranking prompting. In *Findings of the Association for Computational Linguistics: NAACL 2024*, pages 1504–1518, June 2024. doi: 10.18653/v1/2024.findings-naacl.97. URL https://aclanthology.org/2024.findings-naacl.97/.

Inioluwa Deborah Raji, Emily M. Bender, Amandalynne Paullada, Emily Denton, and Alex Hanna. AI and the everything in the whole wide world benchmark. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS 2021)*, 2021. URL https://arxiv.org/abs/2111.15366.

Dieter Rasch, Klaus D. Kubinger, and Karl Moder. The two-sample t test: Pre-testing its assumptions does not pay off. *Statistical Papers*, 52(1):219–231, 2011. doi: 10.1007/s00362-009-0224-x. URL https://doi.org/10.1007/s00362-009-0224-x.

Justine Rochon, Matthias Gondan, and Meinhard Kieser. To test or not to test: Preliminary assessment of normality when comparing two independent samples. *BMC Medical Research Methodology*, 12:81, 2012. doi: 10.1186/1471-2288-12-81. URL https://doi.org/10.1186/1471-2288-12-81.

Gayathri Saranathan, Cong Xu, Mahammad Parwez Alam, Tarun Kumar, Martin Foltin, Soon Yee Wong, and Suparna Bhattacharya. SubLIME: Subset selection via rank correlation prediction for data-efficient LLM evaluation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 30572–30593, July 2025. doi: 10.18653/v1/2025.acl-long.1477. URL https://aclanthology.org/2025.acl-long.1477/.

Kayla Schroeder and Zach Wood-Doughty. Can you trust LLM judgments? Reliability of LLM-as-a-judge, December 2024. URL https://arxiv.org/abs/2412.12509. v2, 18 February 2025.

William R. Schucany and H. K. Tony Ng. Preliminary goodness-of-fit tests for normality do not validate the one-sample Student t. *Communications in Statistics – Theory and Methods*, 35(12): 2275–2286, 2006. doi: 10.1080/03610920600853308. URL https://doi.org/10.1080/03610920600853308.

Joseph P. Simmons, Leif D. Nelson, and Uri Simonsohn. False-positive psychology: Undisclosed flexibility in data collection and analysis allows presenting anything as significant. *Psychological Science*, 22(11):1359–1366, 2011. doi: 10.1177/0956797611417632. URL https://doi.org/10.1177/0956797611417632.

Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters, August 2024. URL https://arxiv.org/abs/2408.03314.

Yifan Song, Guoyin Wang, Sujian Li, and Bill Yuchen Lin. The good, the bad, and the greedy: Evaluation of LLMs should not ignore non-determinism, July 2024. URL https://arxiv.org/abs/2407.10457.

Rickard Stureborg, Dimitris Alikaniotis, and Yoshi Suhara. Large language models are inconsistent and biased evaluators, May 2024. URL https://arxiv.org/abs/2405.01724.

Aman Singh Thakur, Kartik Choudhary, Venkat Srinik Ramayapally, Sankaran Vaidyanathan, and Dieuwke Hupkes. Judging the judges: Evaluating alignment and vulnerabilities in LLMs-as-judges. In *Proceedings of the Fourth Workshop on Generation, Evaluation and Metrics (GEM^2 2025)*, pages 404–430, July 2025. URL https://arxiv.org/abs/2406.12624. arXiv v6, 18 August 2025.

Prasanth Aby Thomas. Microsoft adds multi-model AI to Copilot Researcher, raising accuracy stakes. Computerworld, March 2026. URL https://www.computerworld.com/article/4152323/microsoft-adds-multi-model-ai-to-copilot-researcher-raising-accuracy-stakes.html. Published 31 March 2026; independent press report of Anand [2026]. Accessed 10 September 2026.

Tuhina Tripathi, Manya Wadhwa, Greg Durrett, and Scott Niekum. Pairwise or pointwise? Evaluating feedback protocols for bias in LLM-based evaluation. In *Second Conference on Language Modeling (COLM 2025)*, 2025. URL https://arxiv.org/abs/2504.14716.

Valyu. Introducing DeepResearch, May 2026. URL https://www.valyu.ai/blogs/introducing-deepresearch. Published 5 May 2026 (byline Harvey Yorke). Accessed 10 September 2026; re-checked 26 October 2026.

Willem van der Maden, Malak Sadek, Ziang Xiao, Aske Mottelson, Q. Vera Liao, and Jichen Zhu. Results-actionability gap: Understanding how practitioners evaluate LLM products in the wild, January 2026. URL https://arxiv.org/abs/2604.16304. Published in Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems, pages 1–17, https://doi.org/10.1145/3772318.3791069.

Pat Verga, Sebastian Hofstatter, Sophia Althammer, Yixuan Su, Aleksandra Piktus, Arkady Arkhangorodsky, Minjie Xu, Naomi White, and Patrick Lewis. Replacing judges with juries: Evaluating LLM generations with a panel of diverse models, April 2024. URL https://arxiv.org/abs/2404.18796. v2, 1 May 2024.

Rajan Vivek, Kawin Ethayarajh, Diyi Yang, and Douwe Kiela. Anchor points: Benchmarking models with much fewer examples. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1576–1601, March 2024. doi: 10.18653/v1/2024.eacl-long.95. URL https://aclanthology.org/2024.eacl-long.95/.

Junlin Wang, Jue Wang, Ben Athiwaratkun, Ce Zhang, and James Zou. Mixture-of-agents enhances large language model capabilities, June 2024. URL https://arxiv.org/abs/2406.04692.

Peiyi Wang, Lei Li, Liang Chen, Zefan Cai, Dawei Zhu, Binghuai Lin, Yunbo Cao, Qi Liu, Tianyu Liu, and Zhifang Sui. Large language models are not fair evaluators, May 2023a. URL https://arxiv.org/abs/2305.17926. v2, 30 August 2023; published, with an eleventh author (Lingpeng Kong), in Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2024, pages 9440–9450, https://aclanthology.org/2024.acl-long.511/.

Victor Wang, Michael J. Q. Zhang, and Eunsol Choi. Improving LLM-as-a-judge inference with the judgment distribution. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 23173–23199, November 2025. doi: 10.18653/v1/2025.findings-emnlp.1259. URL https://aclanthology.org/2025.findings-emnlp.1259/.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations (ICLR 2023)*, 2023b. URL https://arxiv.org/abs/2203.11171.

Skyler Wu, Yash Nair, and Emmanuel J. Candès. Efficient evaluation of LLM performance with statistical guarantees, January 2026. URL https://arxiv.org/abs/2601.20251. v3, 8 May 2026.

Yangzhen Wu, Zhiqing Sun, Shanda Li, Sean Welleck, and Yiming Yang. Inference scaling laws: An empirical analysis of compute-optimal inference for problem-solving with language models, 2025. URL https://arxiv.org/abs/2408.00724. v3, 3 March 2025; published as "Inference Scaling Laws: An Empirical Analysis of Compute-Optimal Inference for LLM Problem-Solving" in The Thirteenth International Conference on Learning Representations (ICLR 2025), pages 55824–55845, https://proceedings.iclr.cc/paper_files/paper/2025/hash/8c3caae2f725c8e2a55ecd600563d172-Abstract-Conference.html.

Zongyou Yang, Yinghan Hou, and Xiaokun Yang. When the judge changes, so does the measurement: Auditing LLM-as-judge reliability, July 2026. URL https://arxiv.org/abs/2607.08535. Submitted 9 July 2026.

Seonghyeon Ye, Doyoung Kim, Sungdong Kim, Hyeonbin Hwang, Seungone Kim, Yongrae Jo, James Thorne, Juho Kim, and Minjoon Seo. FLASK: Fine-grained language model evaluation based on alignment skill sets. In *The Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024. URL https://arxiv.org/abs/2307.10928.

Jiayi Yuan, Hao Li, Xinheng Ding, Wenya Xie, Yu-Jhe Li, Wentian Zhao, Kun Wan, Jing Shi, Xia Hu, and Zirui Liu. Understanding and mitigating numerical sources of nondeterminism in LLM inference, June 2025. URL https://arxiv.org/abs/2506.09501. v2, 24 October 2025.

Peiwen Yuan, Shaoxiong Feng, Yiwei Li, Xinglin Wang, Boyuan Pan, Heda Wang, Yao Hu, and Kan Li. BatchEval: Towards human-like text evaluation. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15940–15958, August 2024. doi: 10.18653/v1/2024.acl-long.846. URL https://aclanthology.org/2024.acl-long.846/.

Hangfan Zhang, Zhiyao Cui, Jianhao Chen, Xinrun Wang, Qiaosheng Zhang, Zhen Wang, Dinghao Wu, and Shuyue Hu. Stop overvaluing multi-agent debate – we must rethink evaluation and embrace model heterogeneity, February 2025. URL https://arxiv.org/abs/2502.08788. v3, 21 June 2025.

James Xu Zhao, Bryan Hooi, and See-Kiong Ng. Test-time scaling in reasoning models is not effective for knowledge-intensive tasks yet. In *Third Conference on Language Modeling (COLM 2026)*, 2026a. URL https://arxiv.org/abs/2509.06861. arXiv v3, 6 August 2026.

Jitian Zhao, Changho Shin, Tzu-Heng Huang, Satya Sai Srinath Namburi Gnvv, and Frederic Sala. CARE: Confounder-aware aggregation for reliable LLM evaluation. In *Proceedings of the 43rd International Conference on Machine Learning (ICML 2026)*, volume 306 of *Proceedings of Machine Learning Research*, pages 162114–162150, 2026b. URL https://arxiv.org/abs/2603.00039.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*, pages 46595–46623, 2023. URL https://arxiv.org/abs/2306.05685.

Joey Zhong, Hao Zhang, Clare Southern, Jeremy Yang, Thomas Wang, Kate Jung, Shu Zhang, Denis Yarats, Johnny Ho, and Jerry Ma. DRACO: a cross-domain benchmark for deep research accuracy, completeness, and objectivity, February 2026. URL https://arxiv.org/abs/2602.11685. Perplexity AI. Submitted 12 February 2026.